

Balanced Trees

Part Two

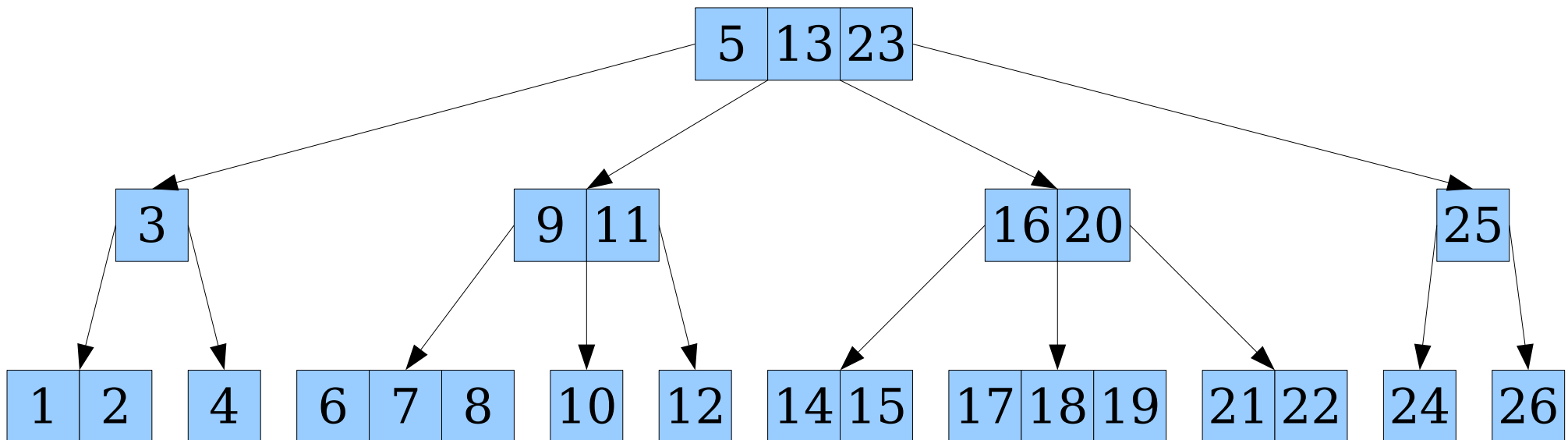
Outline for Today

- ***Red/Black Trees***
 - Using our isometry!
- ***Tree Rotations***
 - A key primitive in restructuring trees.
- ***Augmented Binary Search Trees***
 - Leveraging red/black trees.

Recap from Last Time

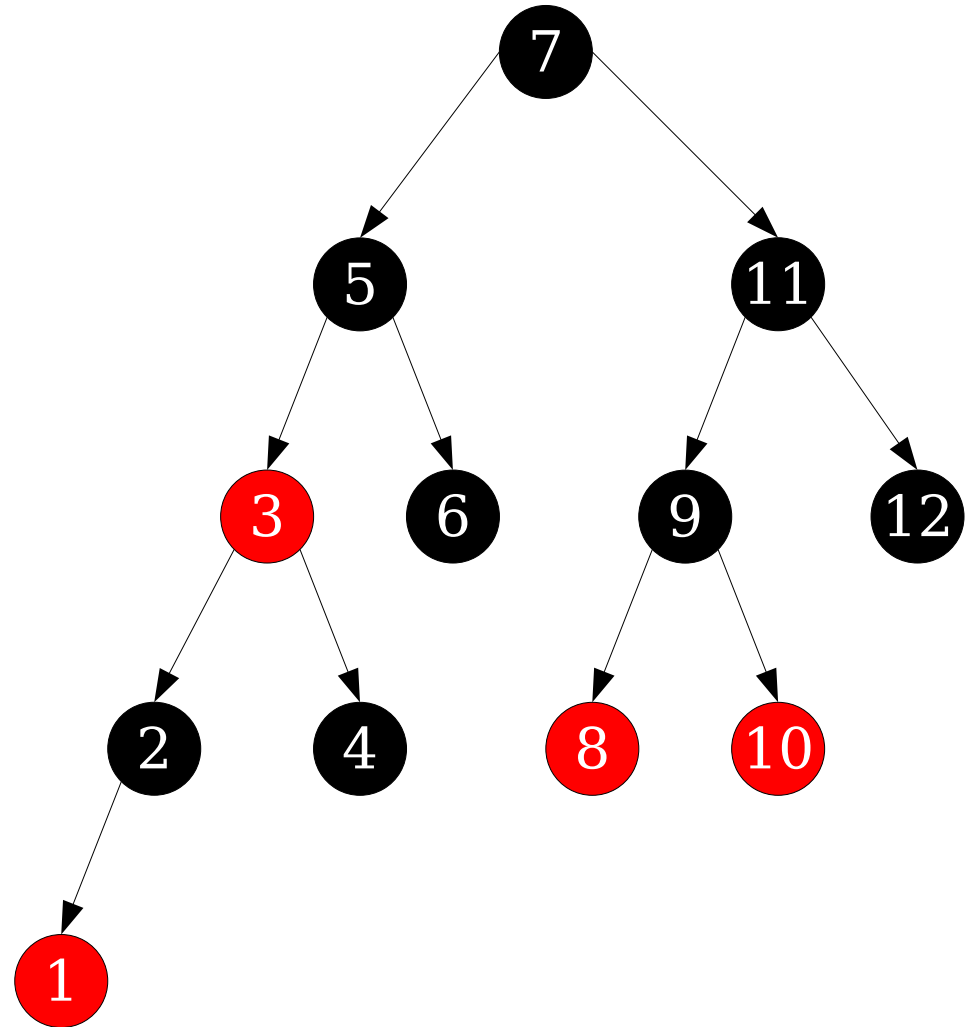
2-3-4 Trees

- A **2-3-4 tree** is a multiway search tree where
 - every node has 1, 2, or 3 keys,
 - any non-leaf node with k keys has exactly $k+1$ children, and
 - all leaves are at the same depth.
- To insert a key, place it in a leaf. If out of space, split the leaf and kick the median key one level higher, repeating this process.



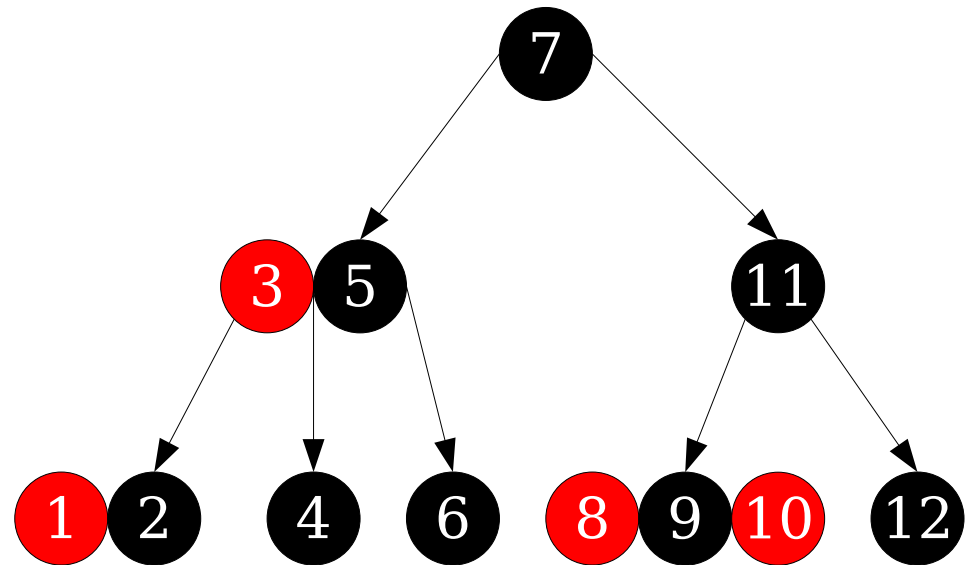
Red/Black Trees

- A **red/black tree** is a BST with the following properties:
 - Every node is either red or black.
 - The root is black.
 - No red node has a red child.
 - Every root-null path in the tree passes through the same number of black nodes.



Red/Black Trees

- A **red/black tree** is a BST with the following properties:
 - Every node is either red or black.
 - The root is black.
 - No red node has a red child.
 - Every root-null path in the tree passes through the same number of black nodes.
- After we hoist red nodes into their parents:
 - Each “meta node” has 1, 2, or 3 keys in it. (No red node has a red child.)
 - Each “meta node” is either a leaf or has one more child than key. (Root-null path property.)
 - Each “meta leaf” is at the same depth. (Root-null path property.)



***This is a
2-3-4 tree!***

New Stuff!

Data Structure Isometries

- Red/black trees are an *isometry* of 2-3-4 trees; they represent the structure of 2-3-4 trees in a different way.
- That gives us some really easy theorems basically for free.
- ***Theorem:*** The maximum height of a red/black tree with n nodes is $O(\log n)$.

Data Structure Isometries

- Red/black trees are an *isometry* of 2-3-4 trees; they represent the structure of 2-3-4 trees in a different way.
- That gives us some really easy theorems basically for free.
- ***Theorem:*** The maximum height of a red/black tree with n nodes is $O(\log n)$.

Explain why, using the isometry.

Formulate a hypothesis!

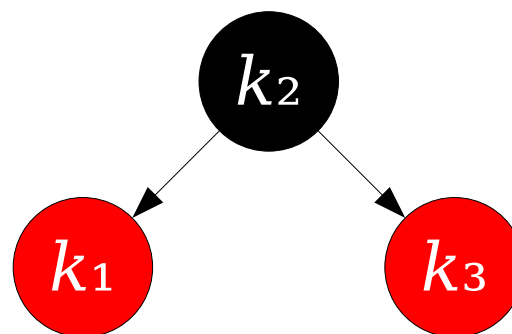
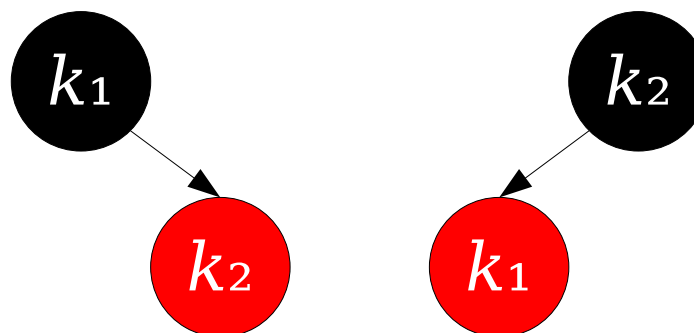
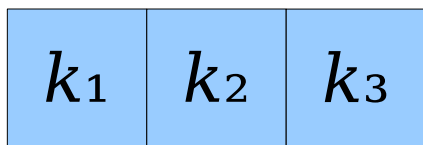
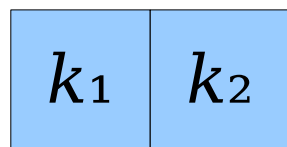
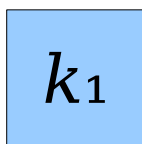
Data Structure Isometries

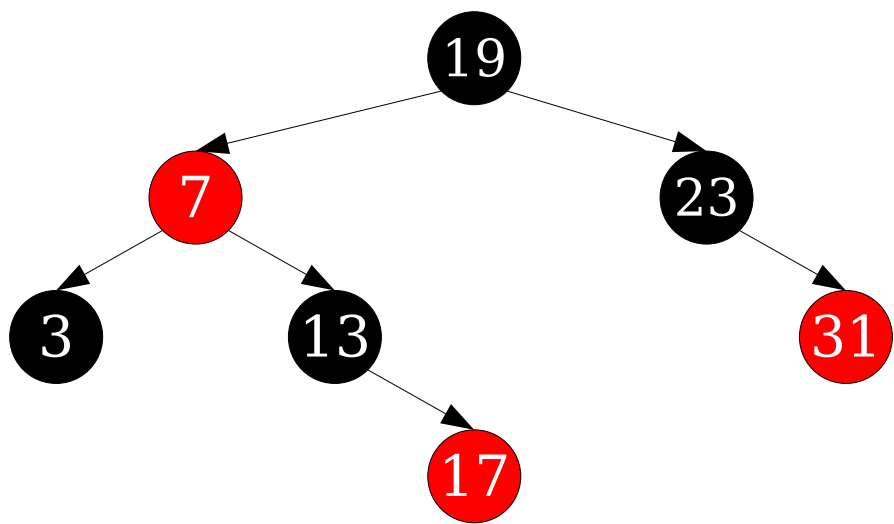
- Red/black trees are an **isometry** of 2-3-4 trees; they represent the structure of 2-3-4 trees in a different way.
- That gives us some really easy theorems basically for free.
- **Theorem:** The maximum height of a red/black tree with n nodes is $O(\log n)$.
- **Proof idea:** Pulling red nodes into their parents forms a 2-3-4 tree with n keys, which has height $O(\log n)$. Undoing this at most doubles the height of the tree. ■-ish

Exploring the Isometry

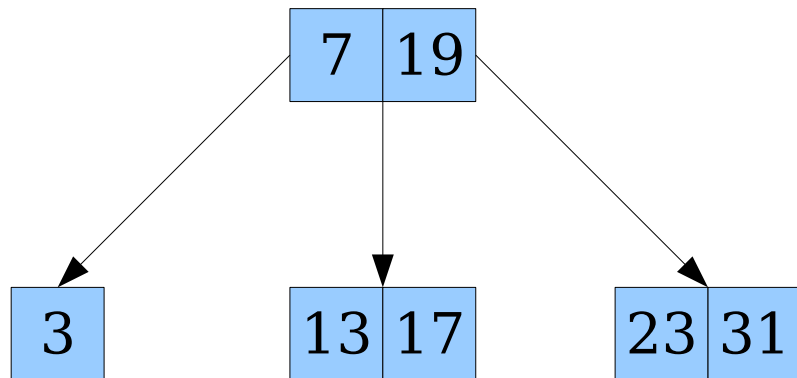
- Nodes in a 2-3-4 tree are classified into types based on the number of children they can have.
 - **2-nodes** have one key (two children).
 - **3-nodes** have two keys (three children).
 - **4-nodes** have three keys (four children).
- How might these nodes be represented?

Exploring the Isometry

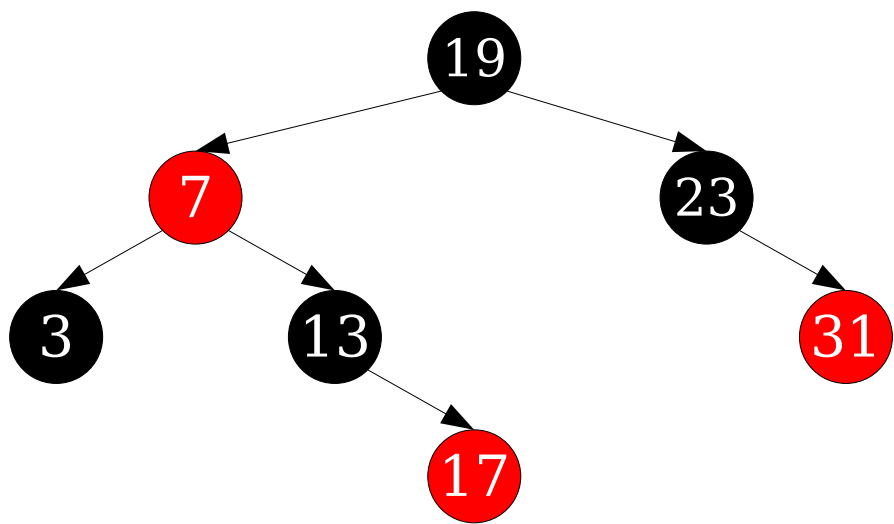




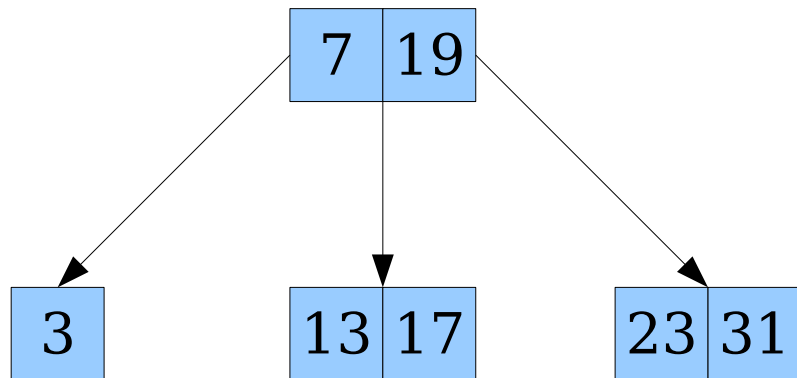
add
5



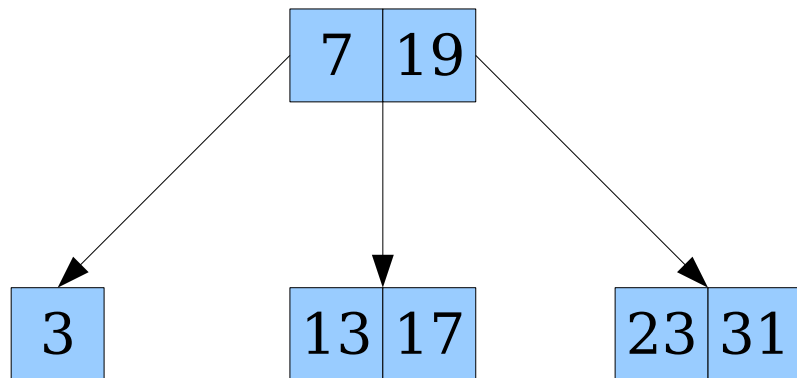
add
5

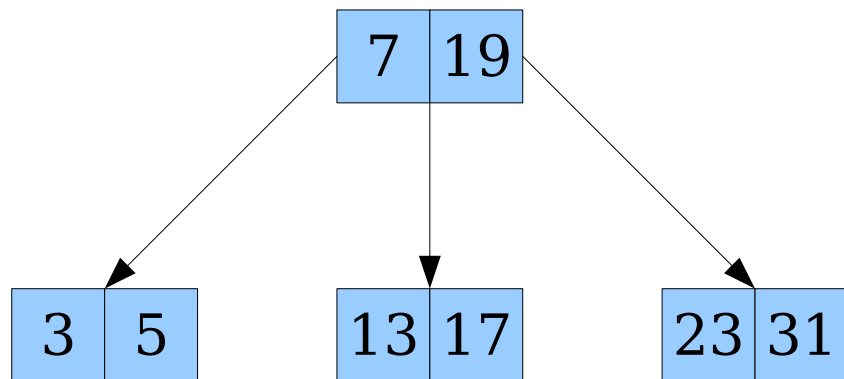
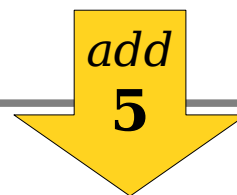
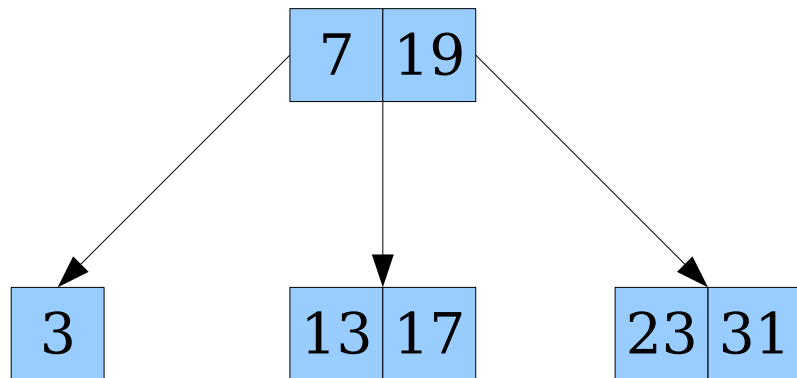
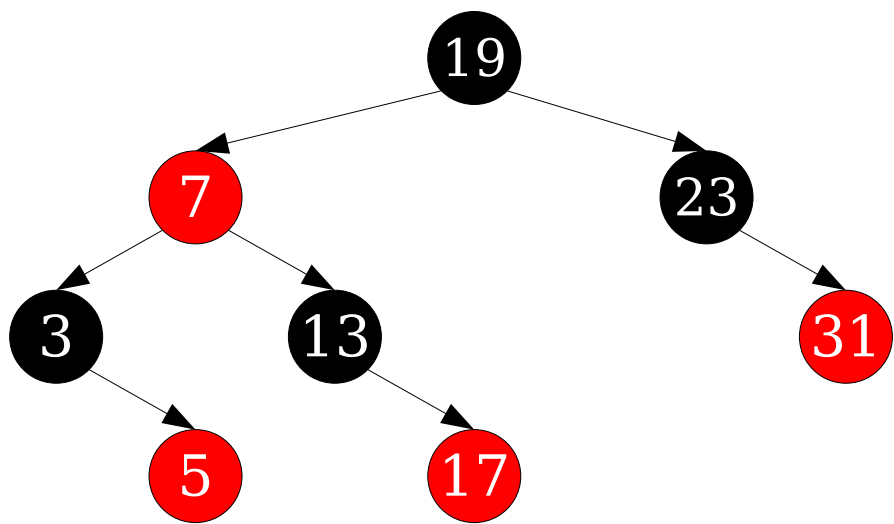
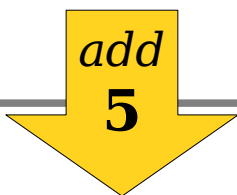
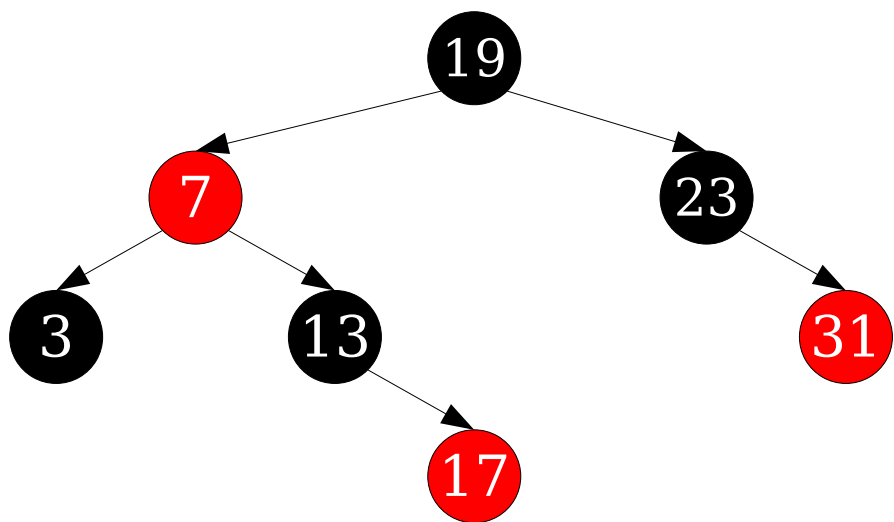


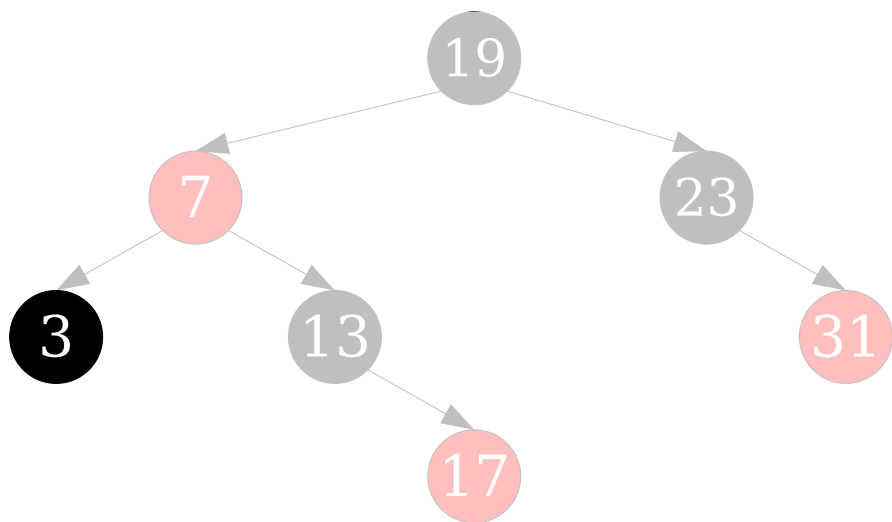
add
5



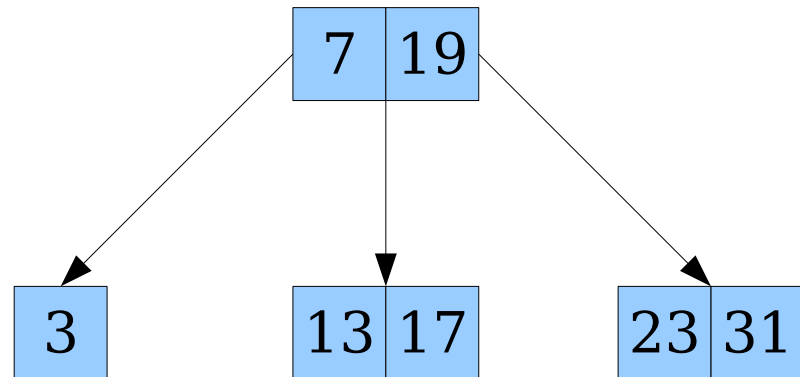
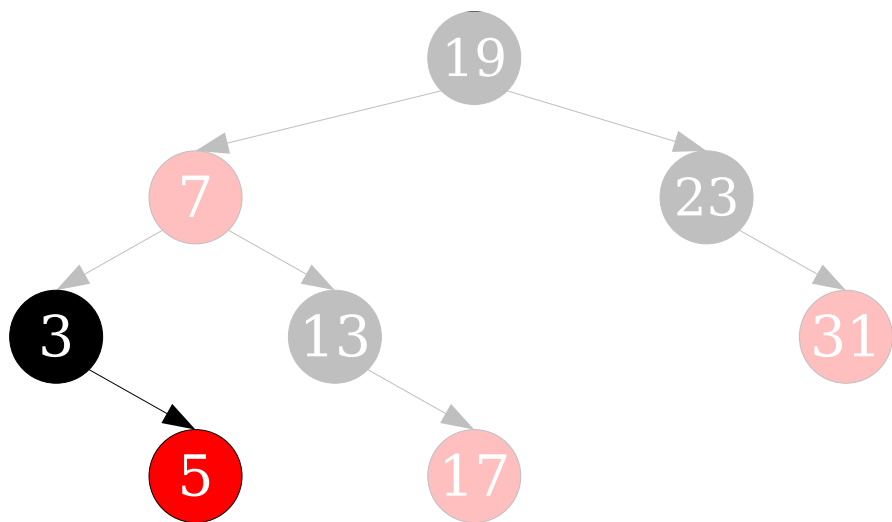
add
5



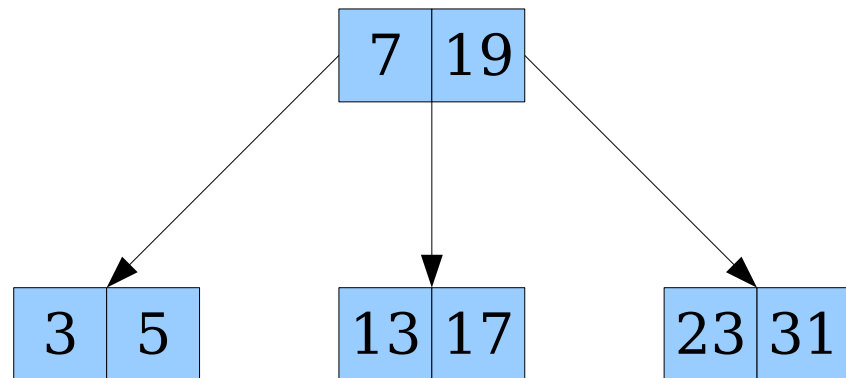


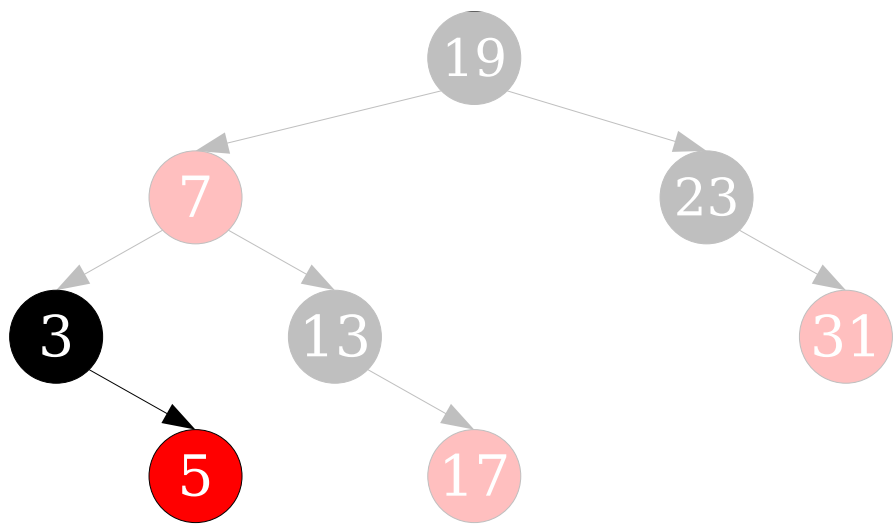
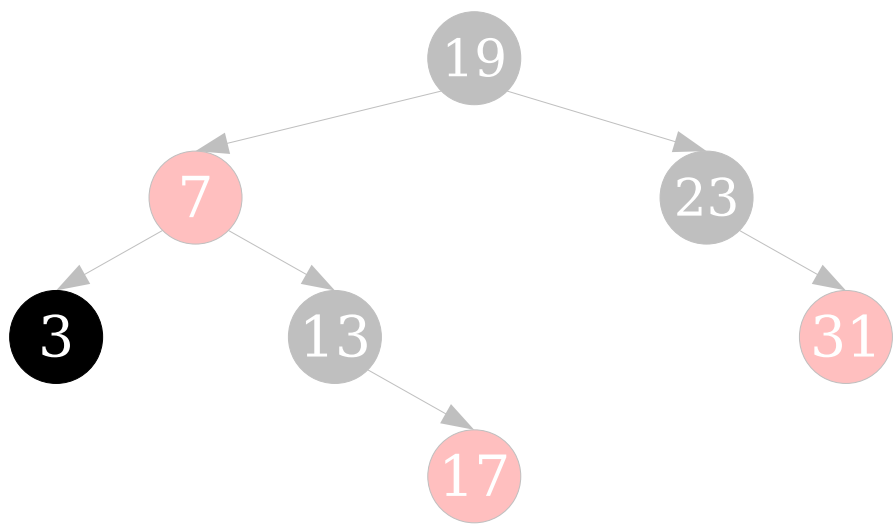


add
5

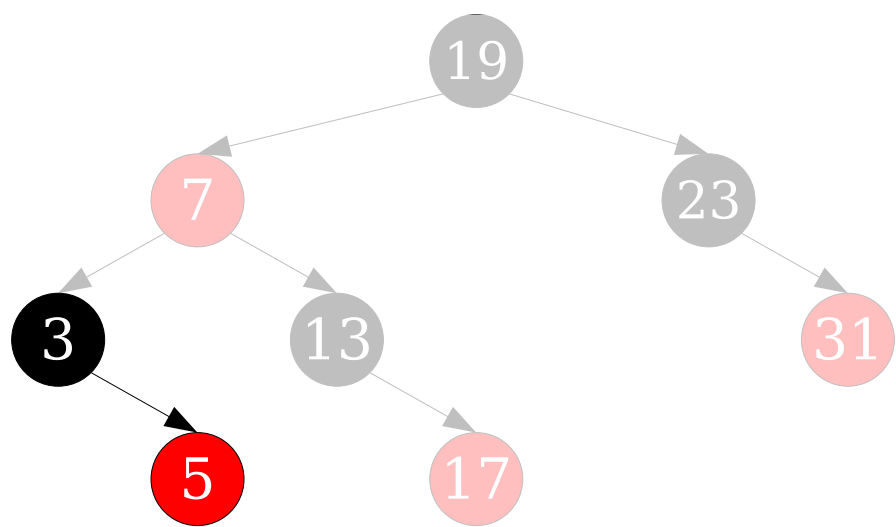
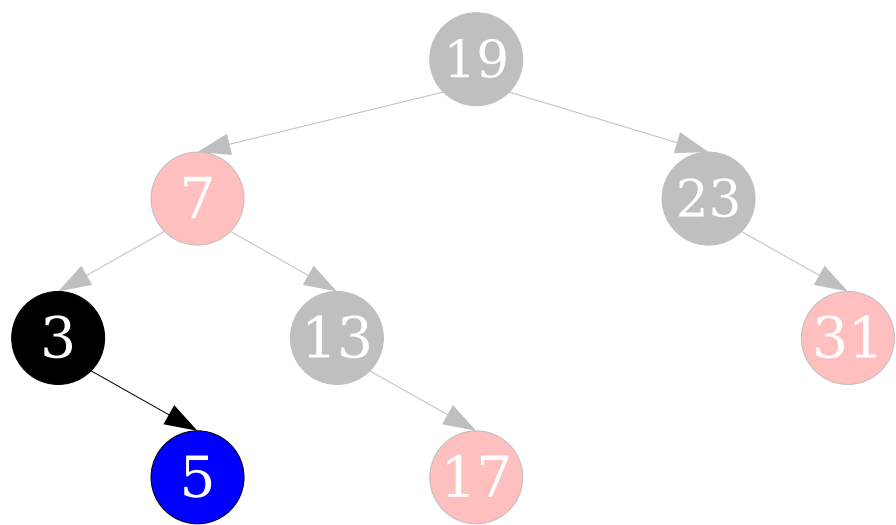


add
5

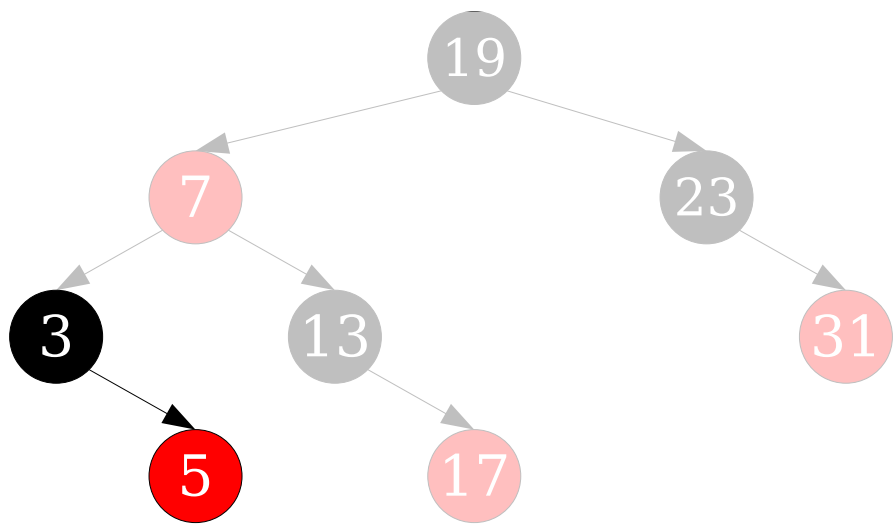
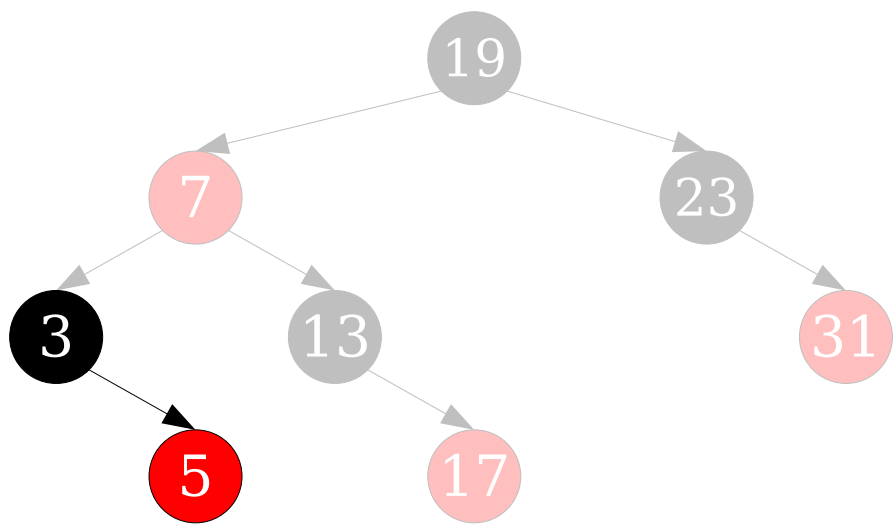




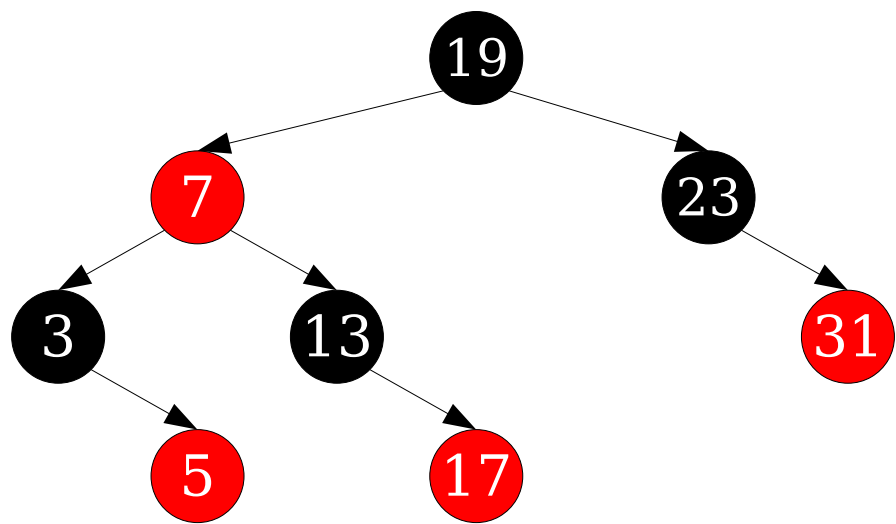
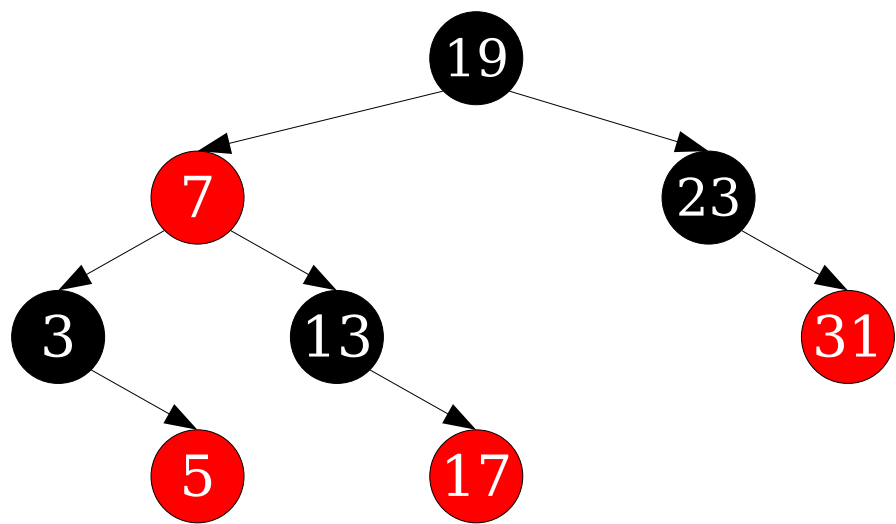
Goal



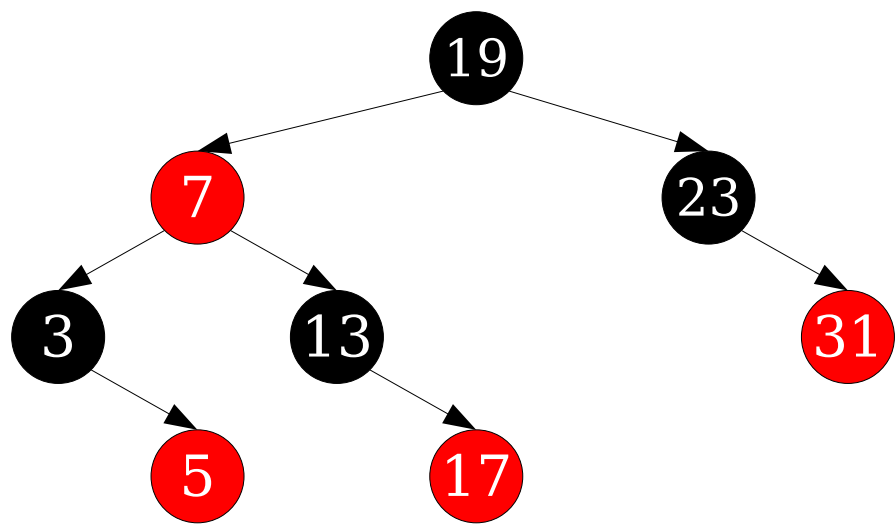
Goal

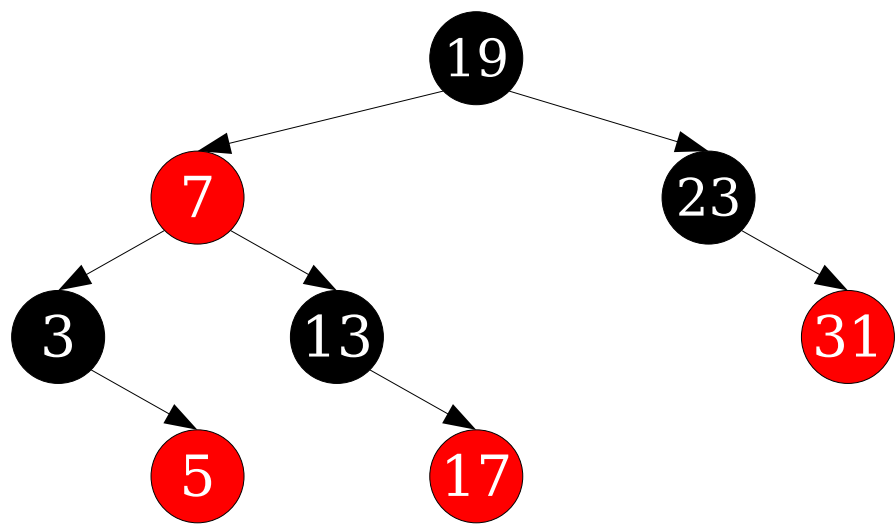


Goal

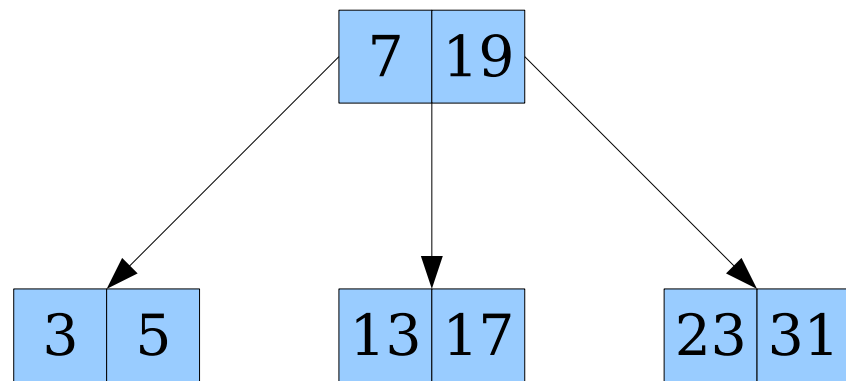


Goal

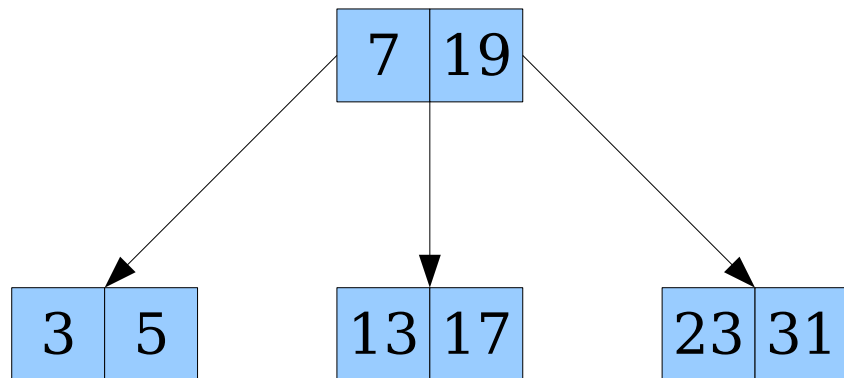
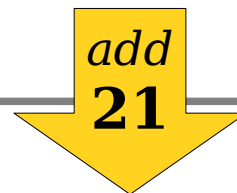
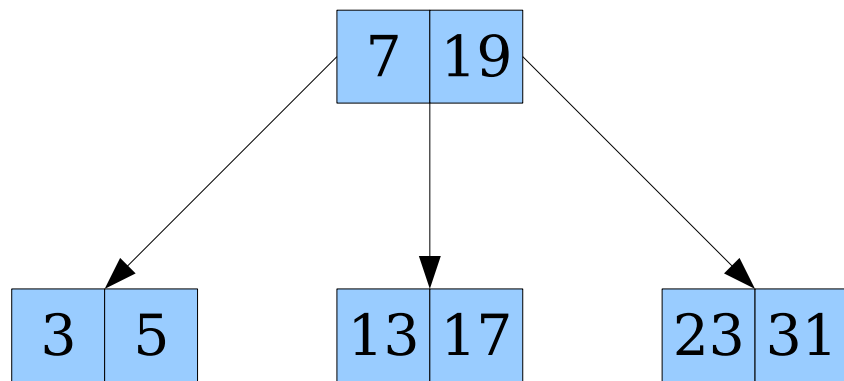
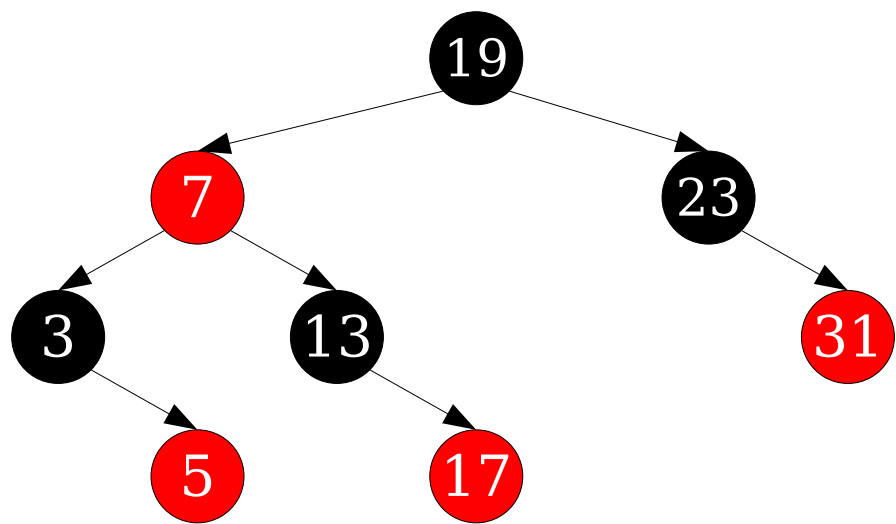


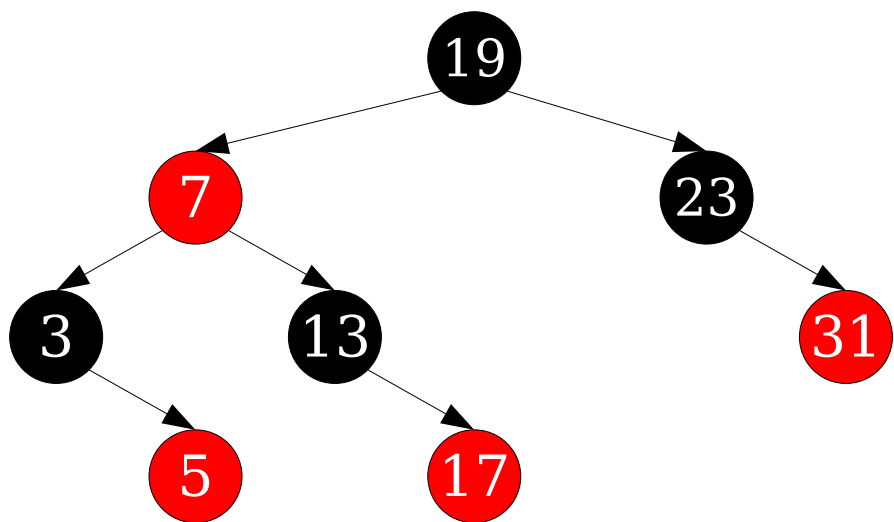


add
21

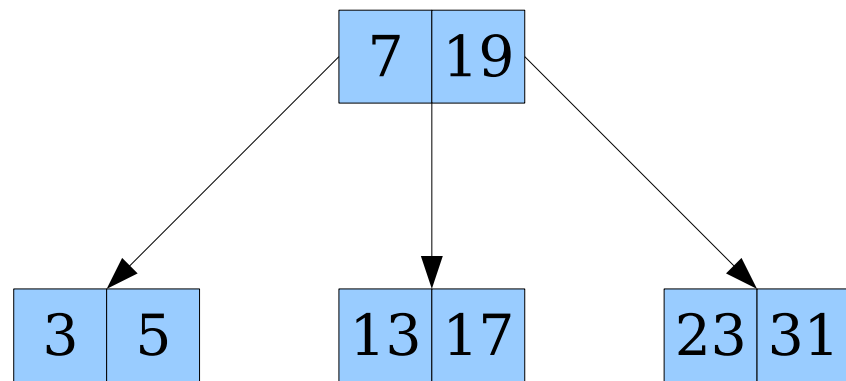
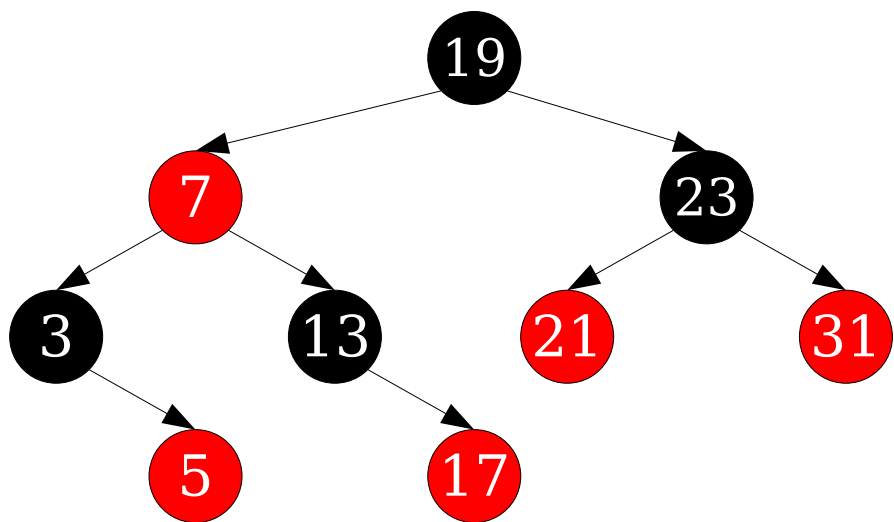


add
21

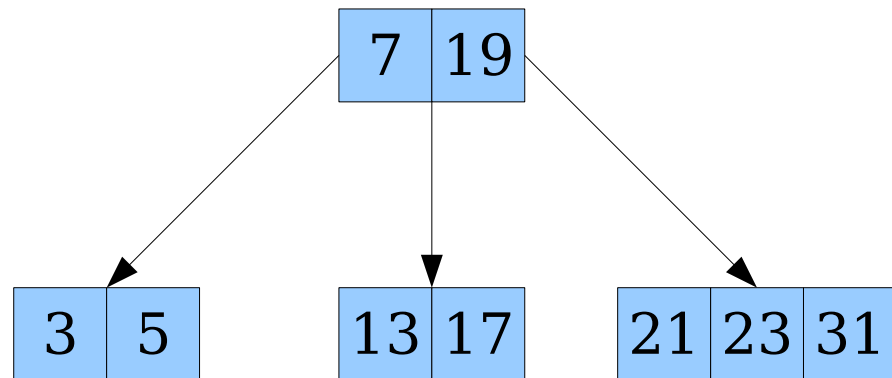


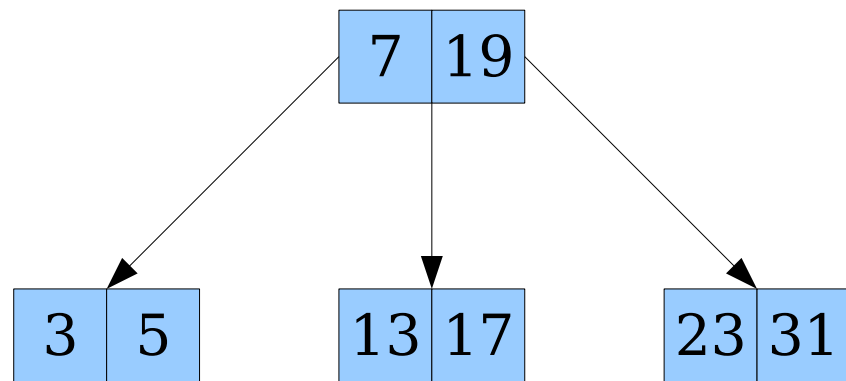
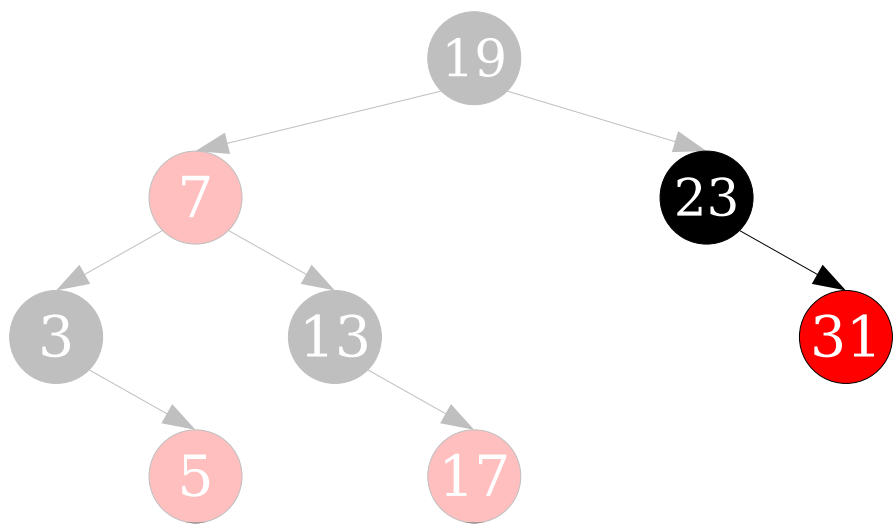


add
21



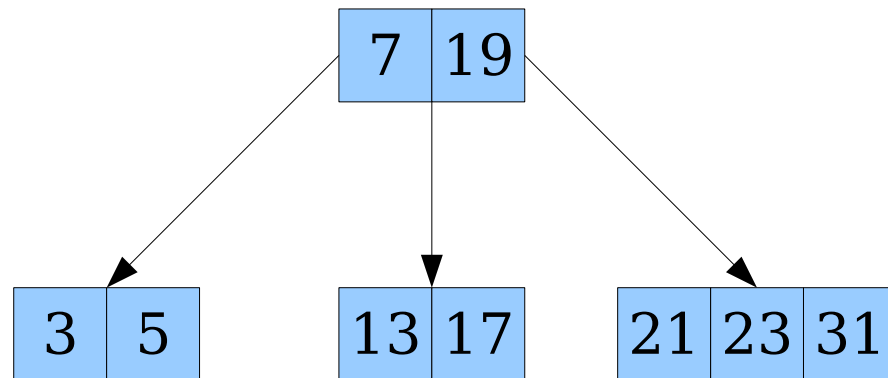
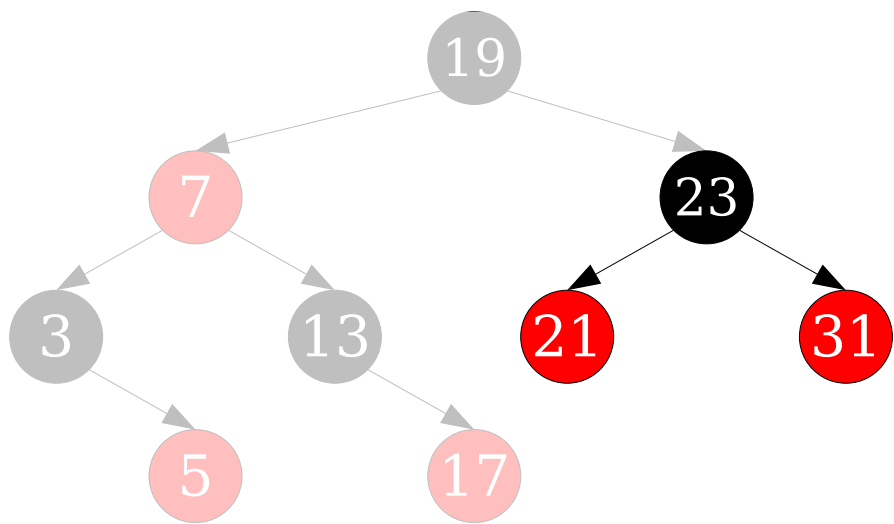
add
21

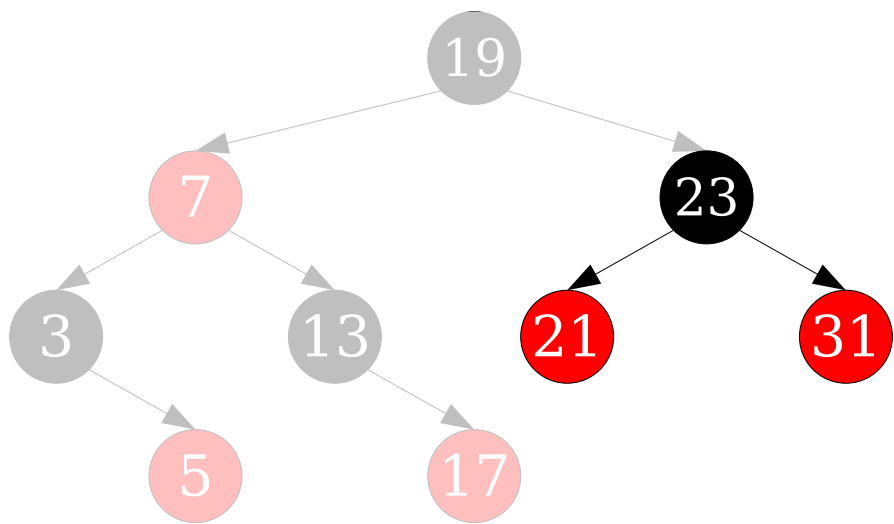
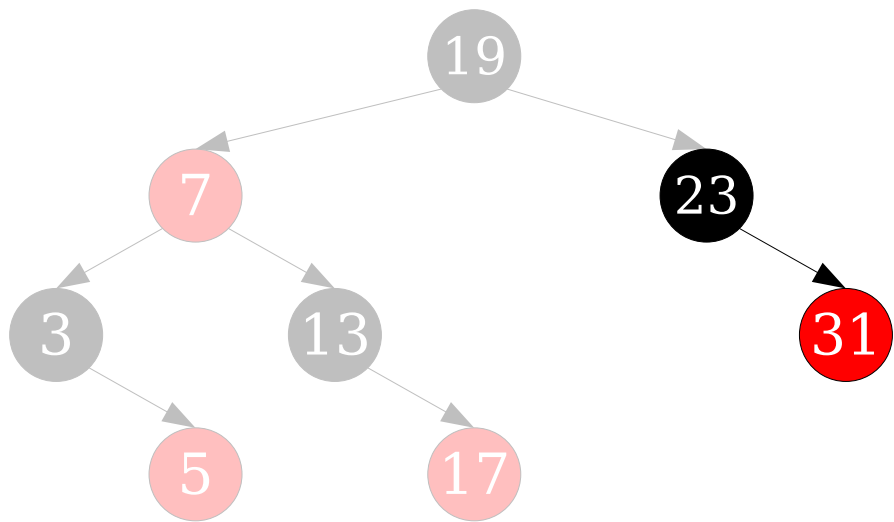




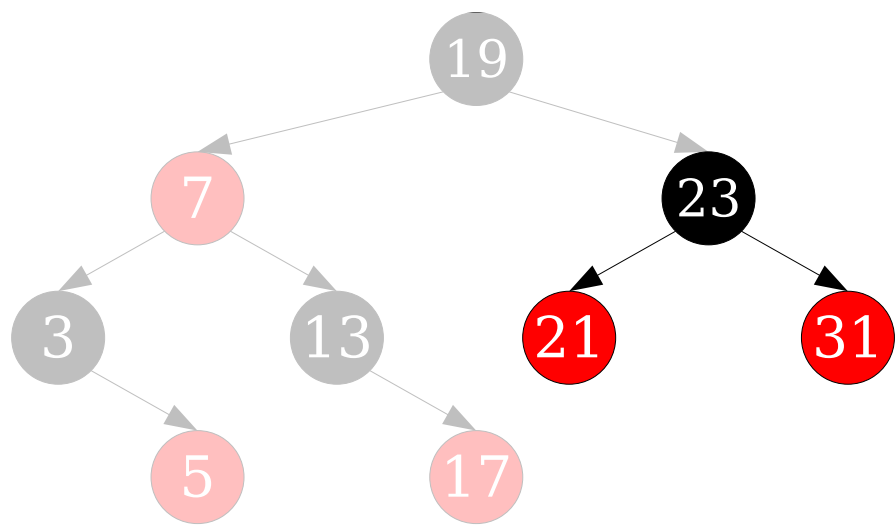
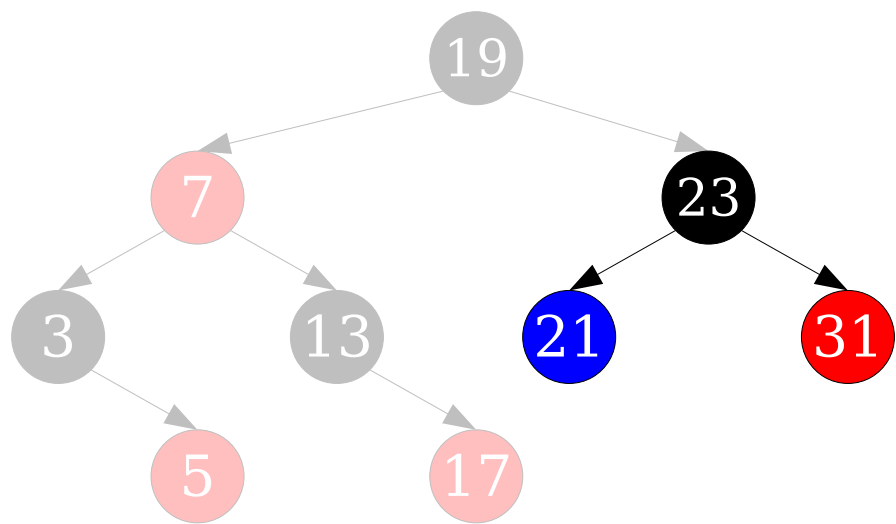
add
21

add
21

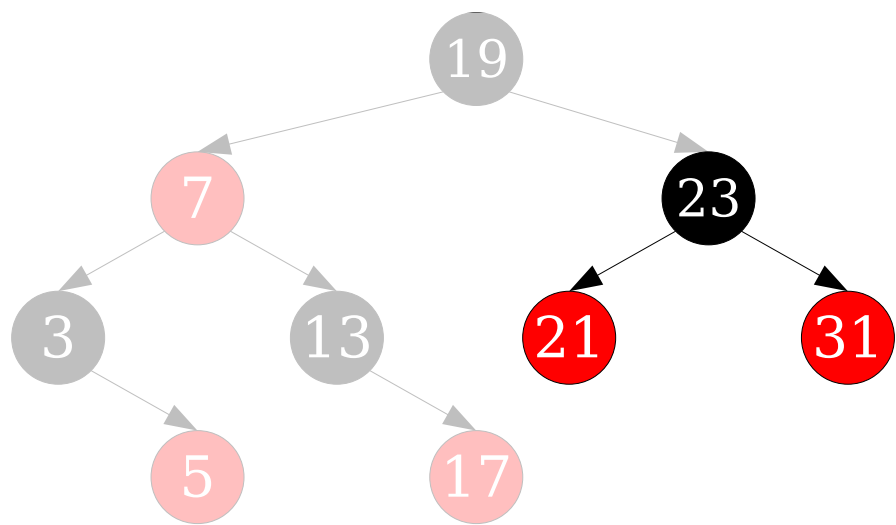
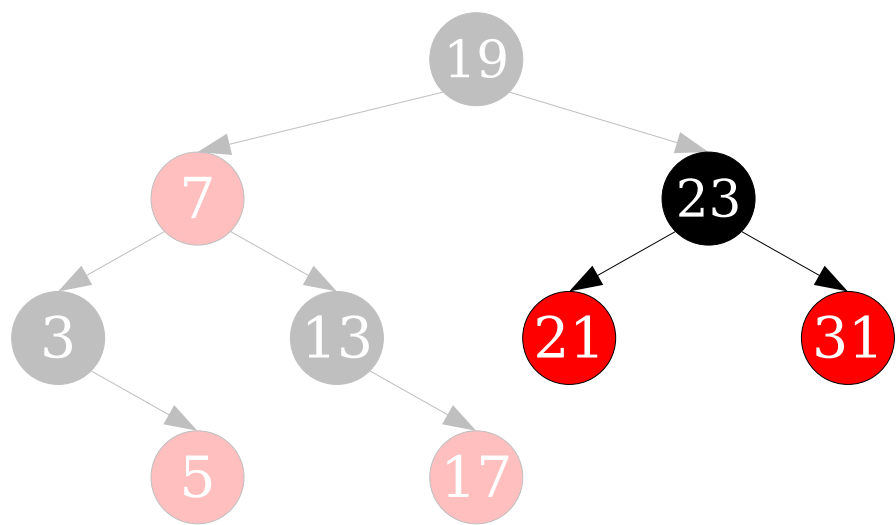




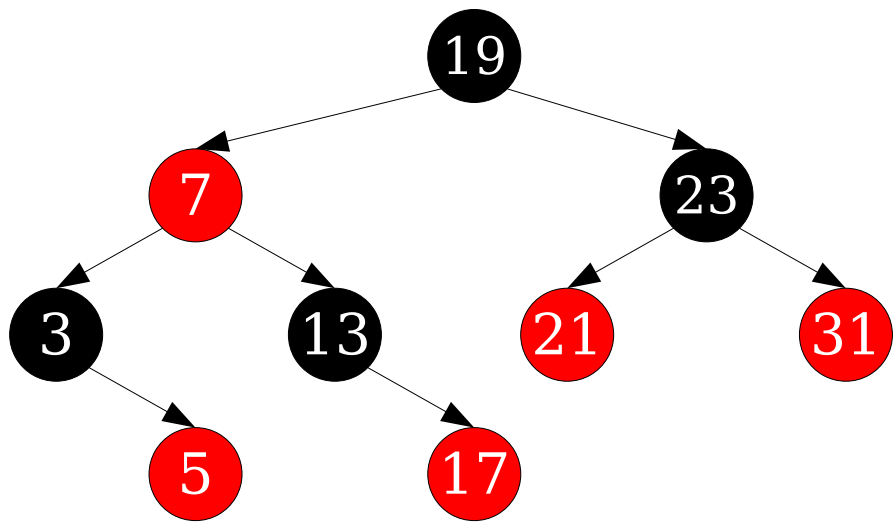
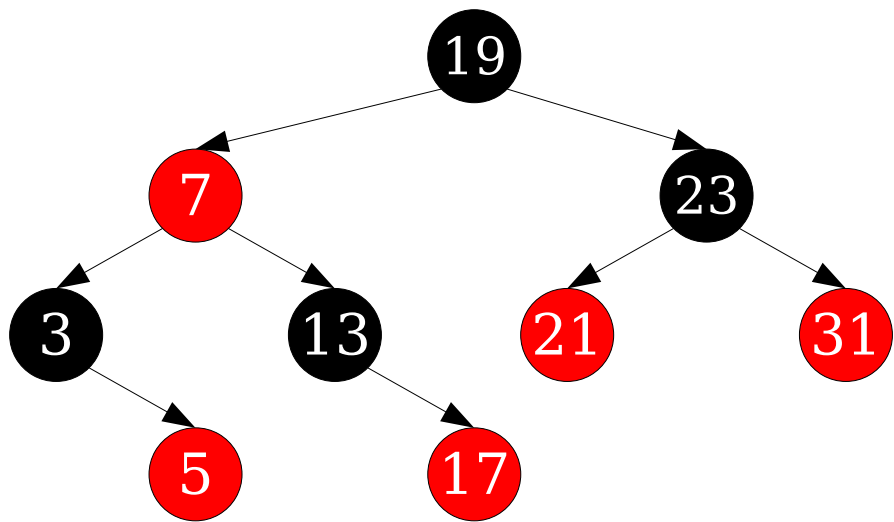
Goal



Goal



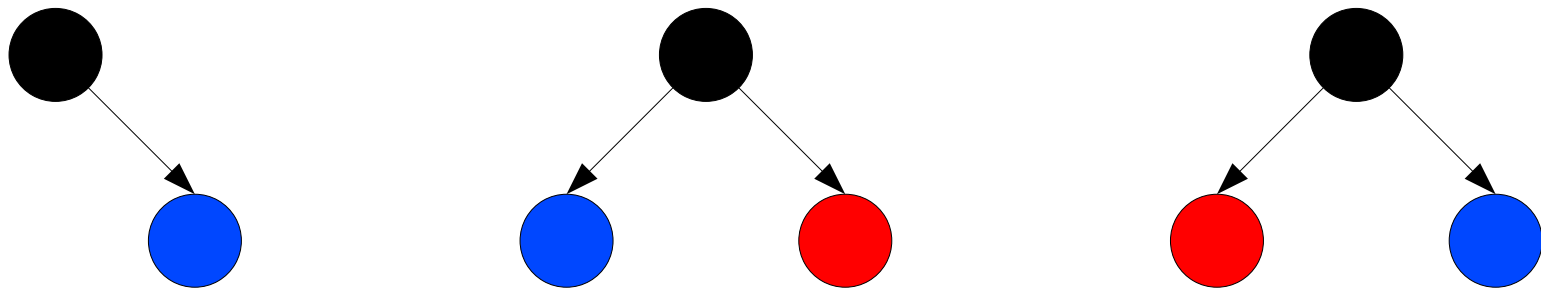
Goal

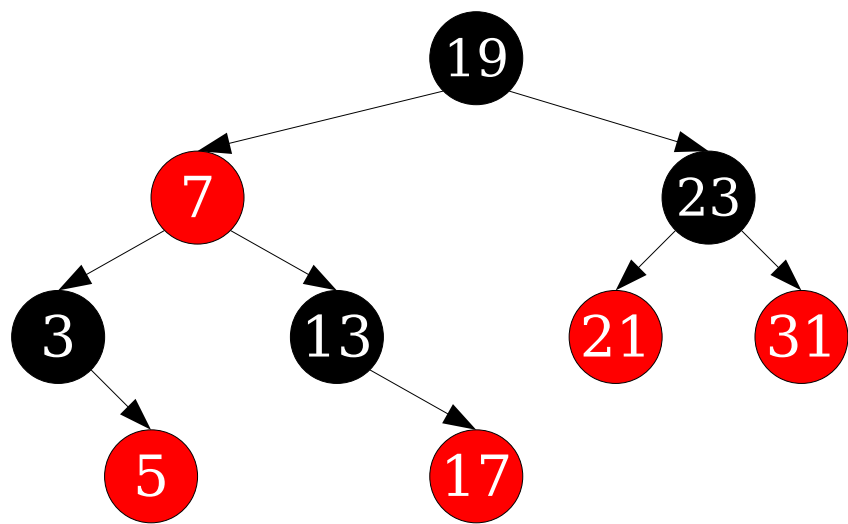


Goal

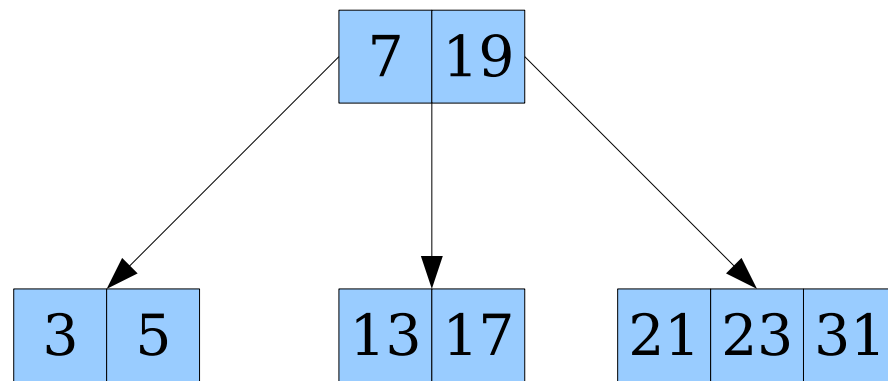
Red/Black Tree Insertion

- **Rule #1:** When inserting a node, if its parent is black, make the node red and stop.
- **Justification:** This simulates inserting a key into an existing 2-node or 3-node.

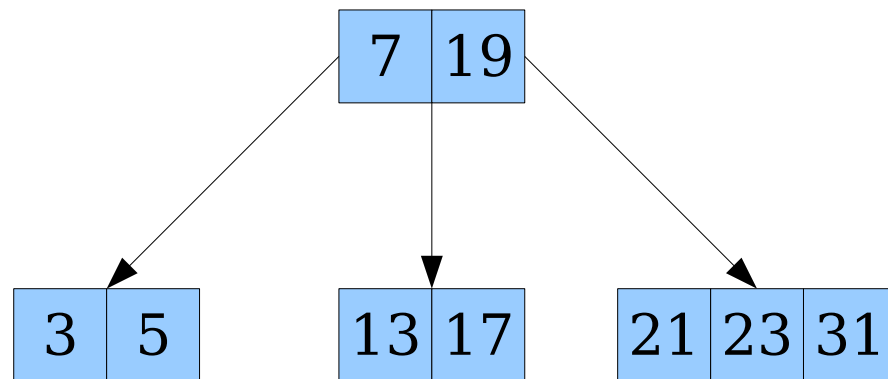


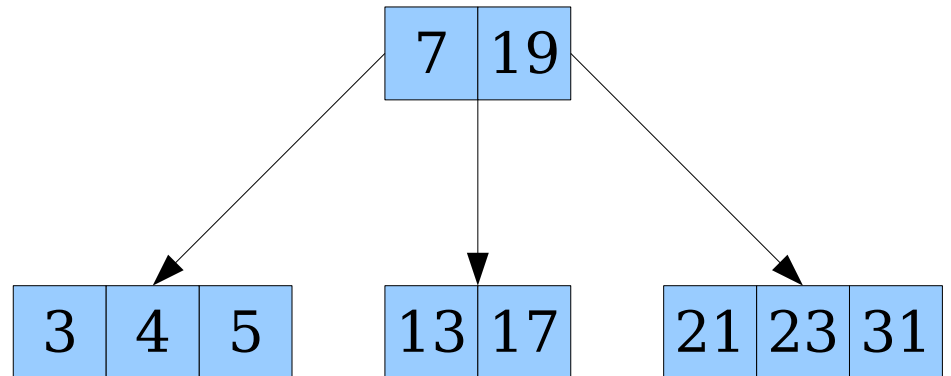
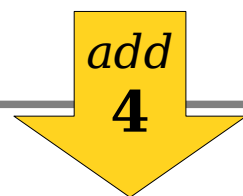
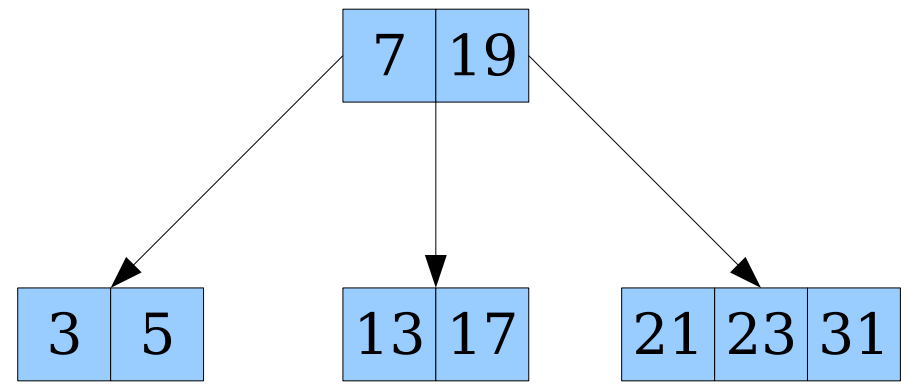
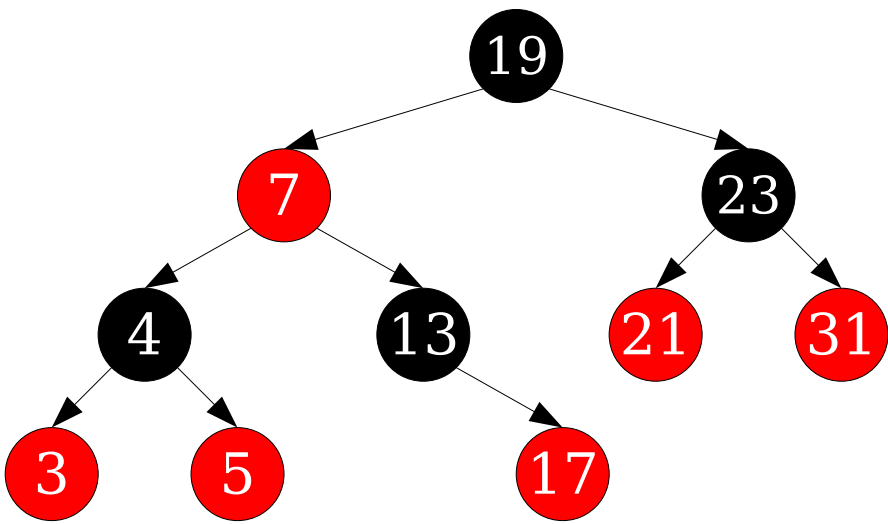
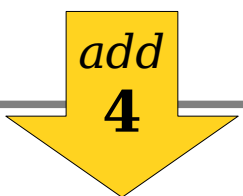
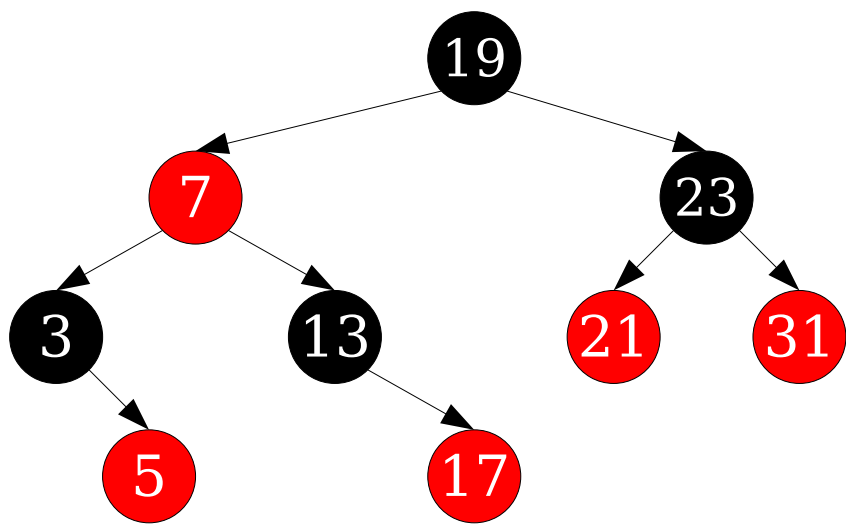


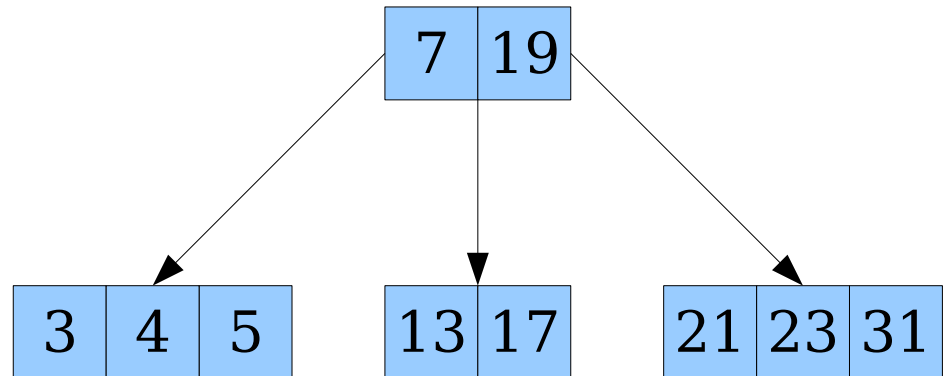
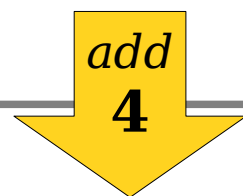
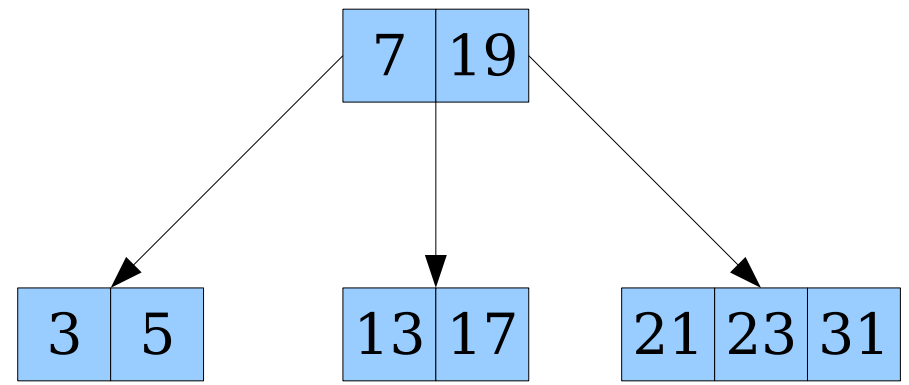
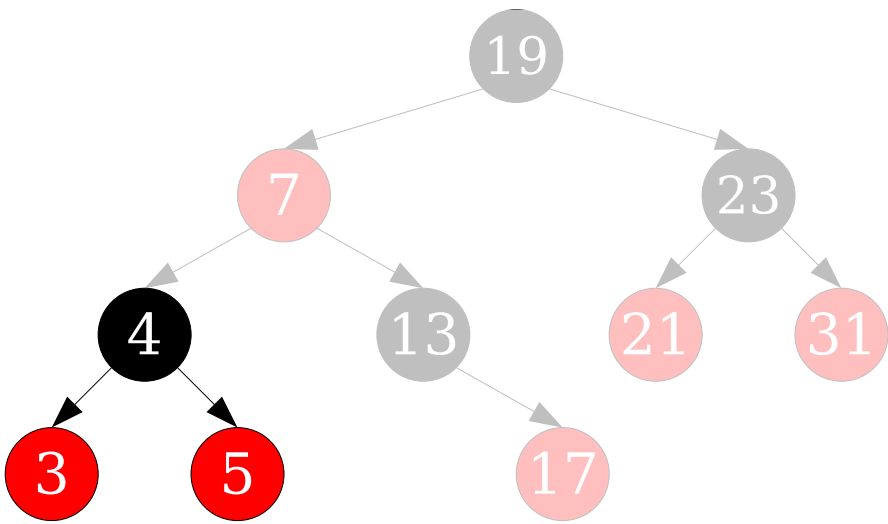
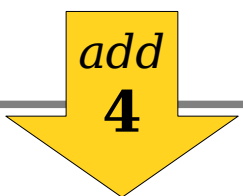
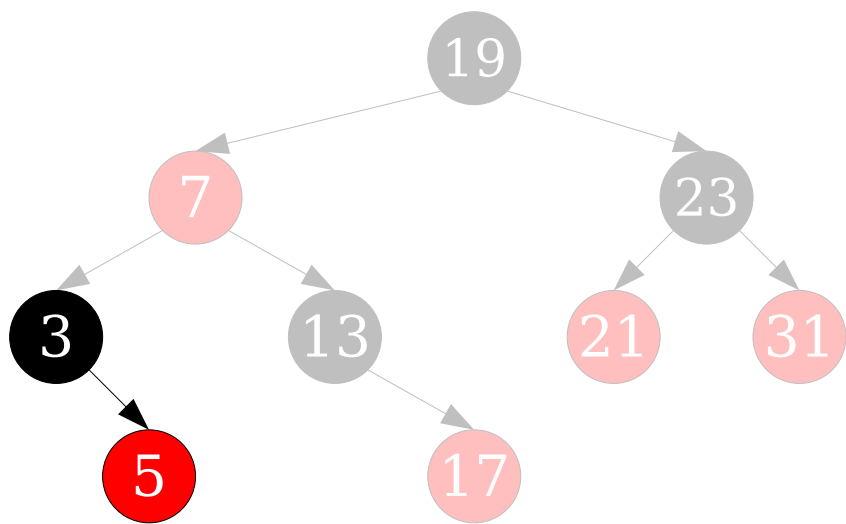
add
4

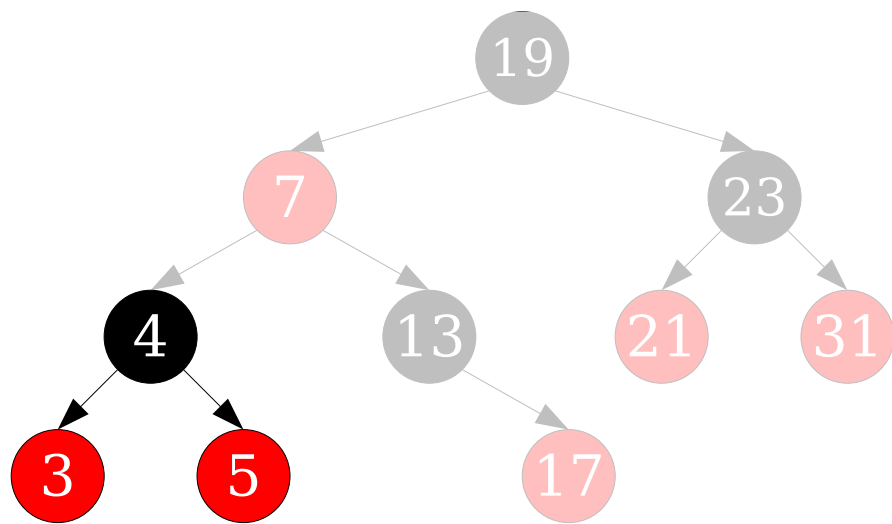
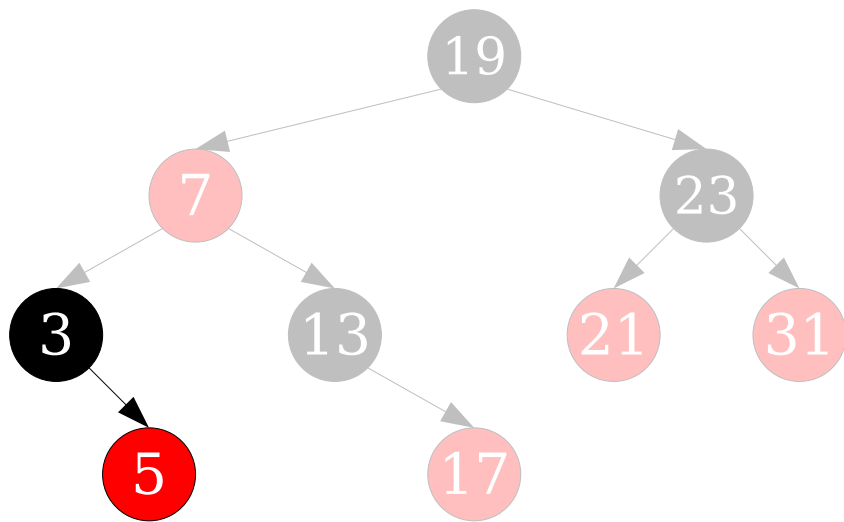


add
4

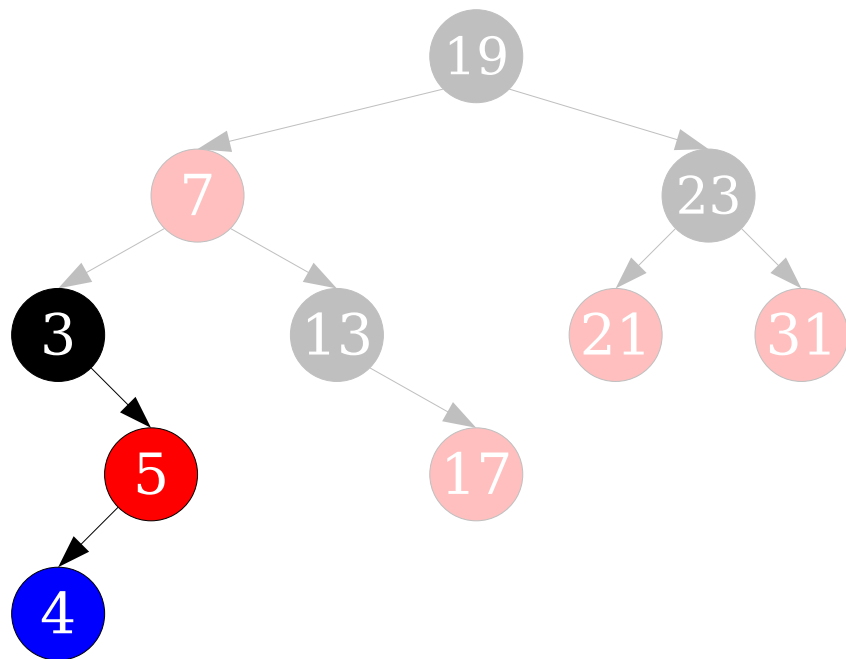




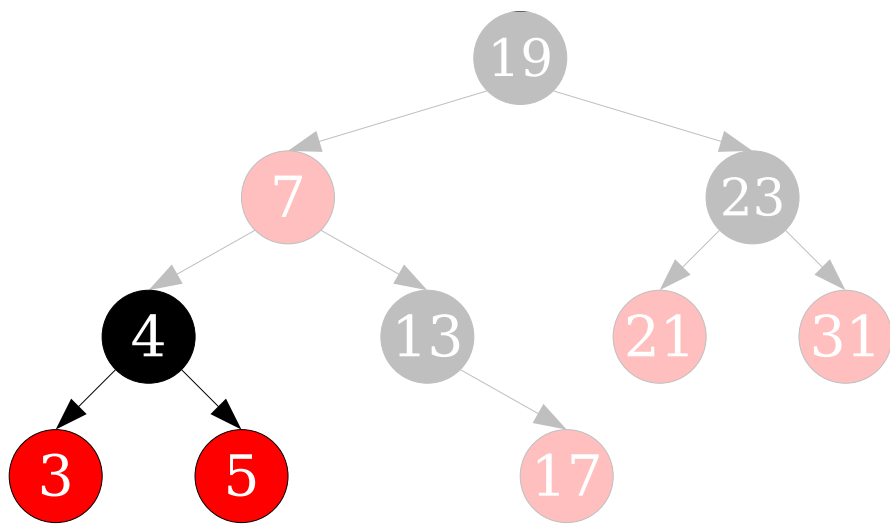




Goal

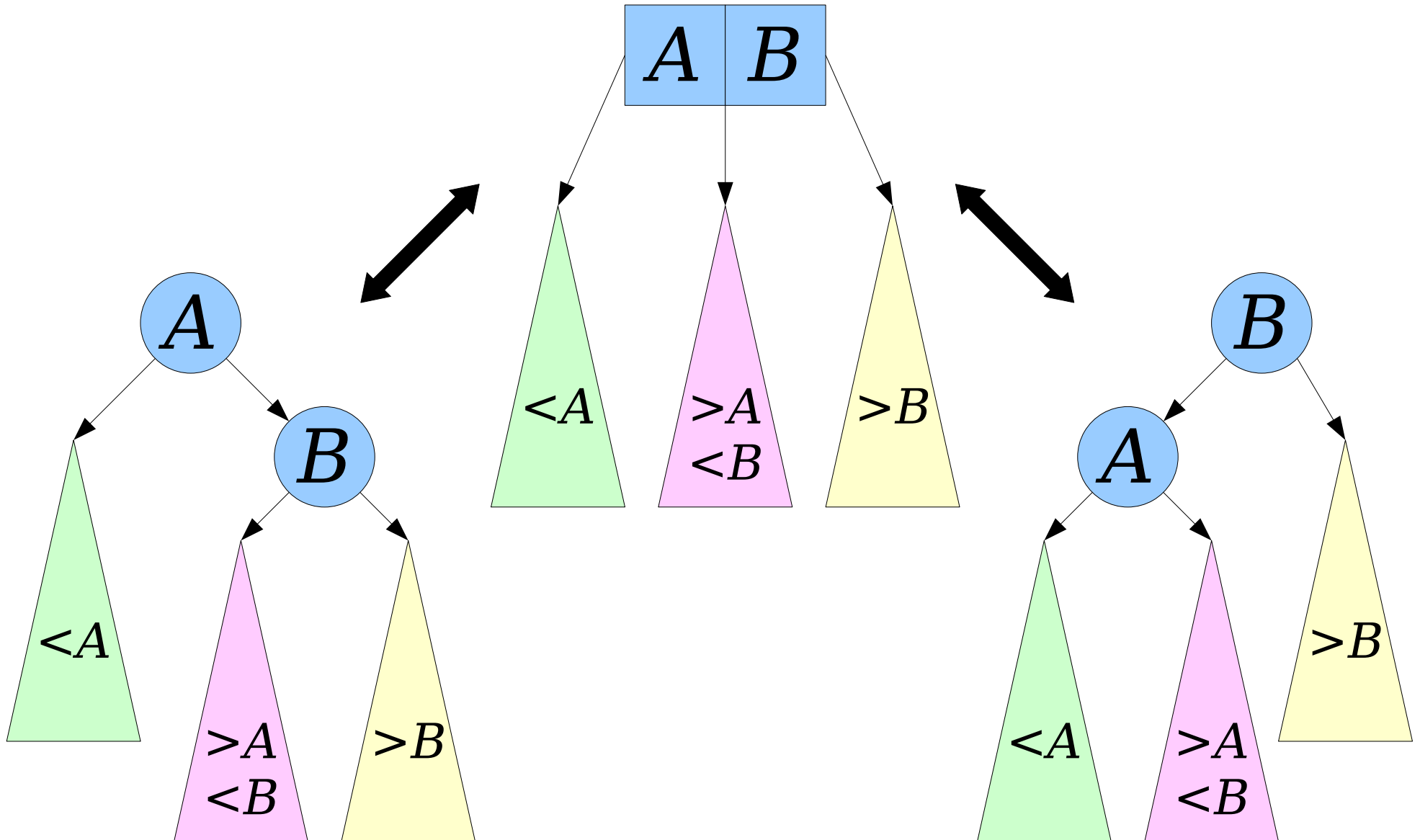


We need to move nodes around in a binary search tree. How do we do this?

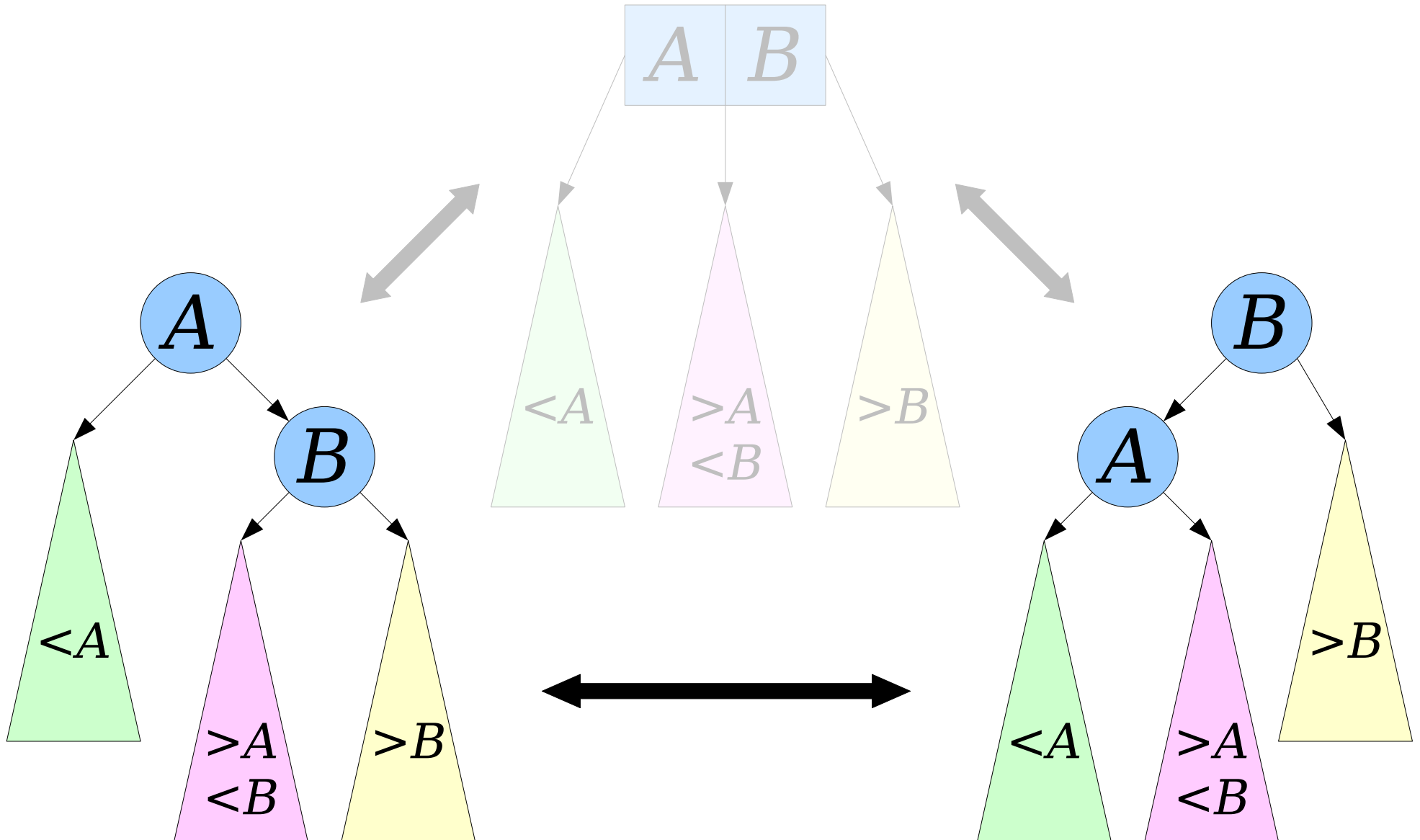


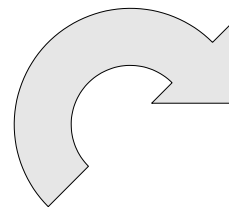
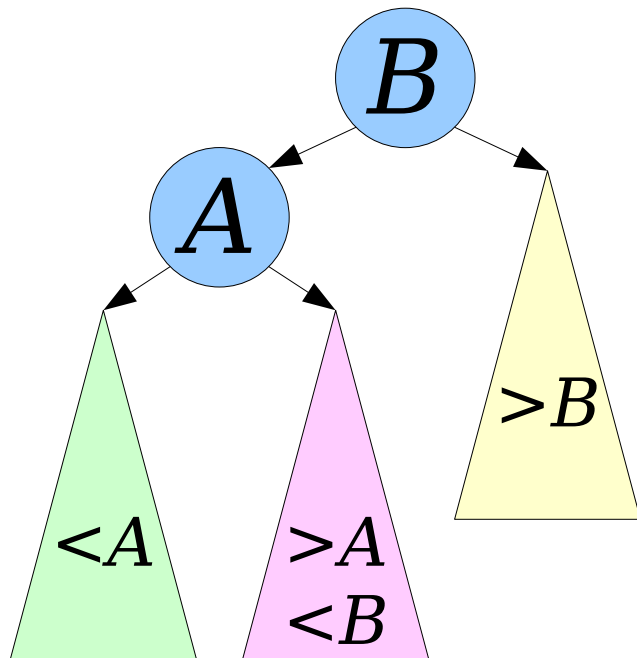
Goal

Tree Rotations

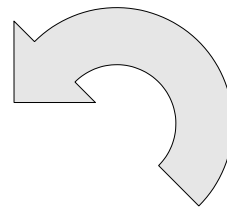
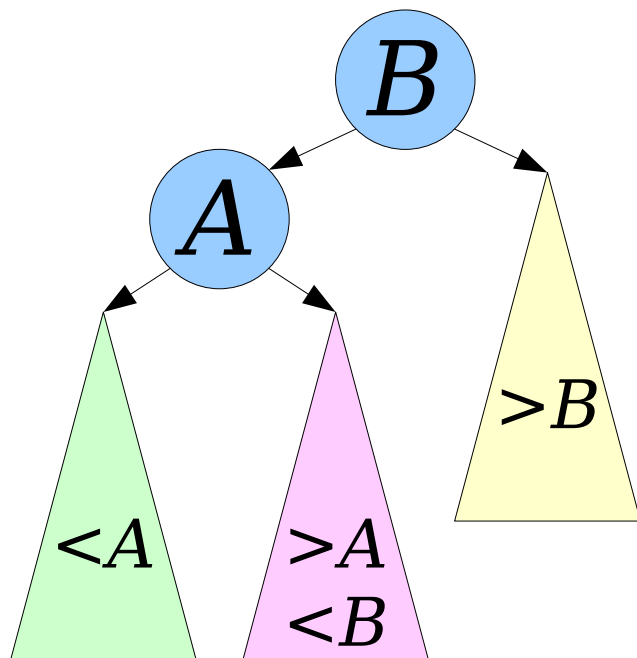
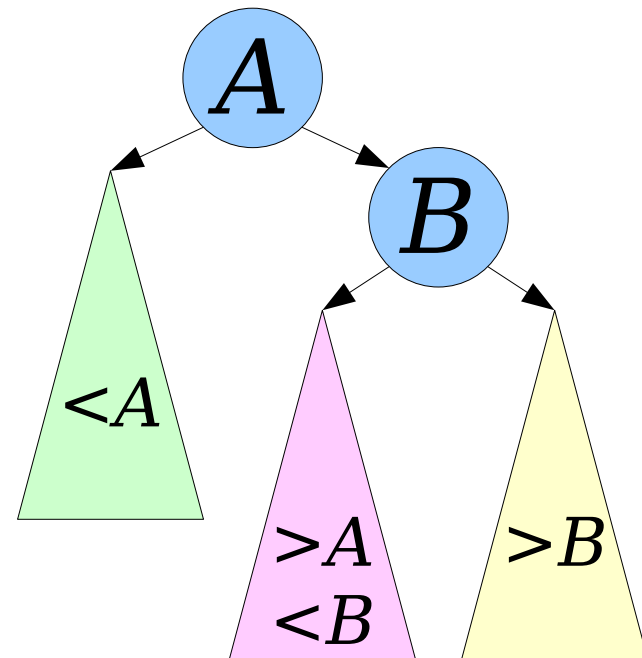


Tree Rotations

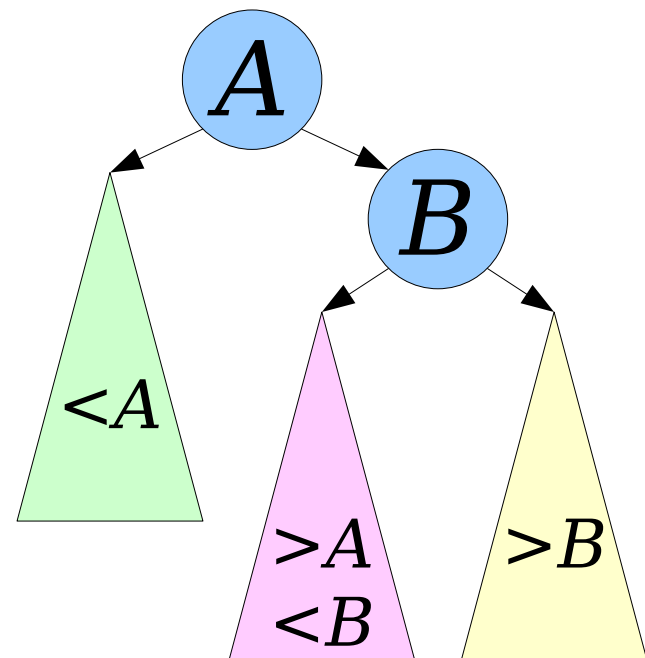


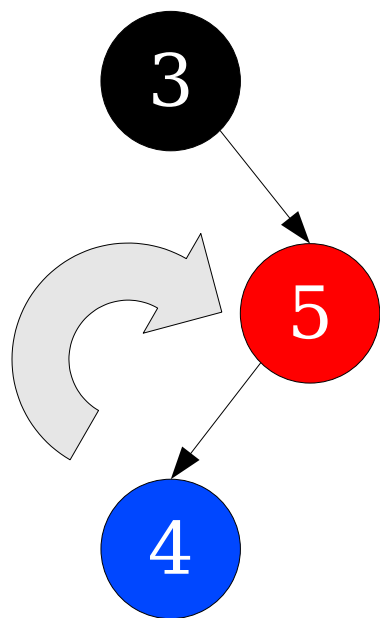


*Rotate
Right*

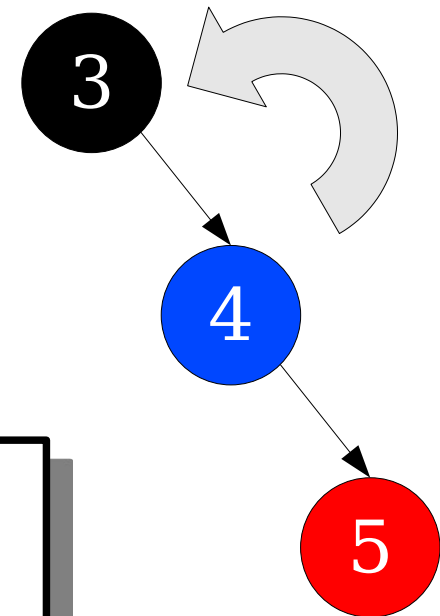
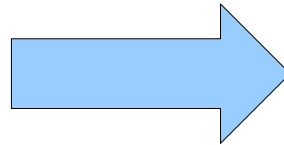


*Rotate
Left*

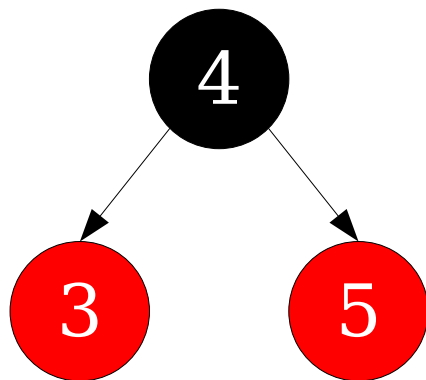
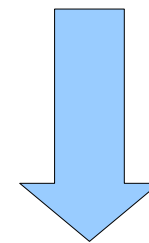




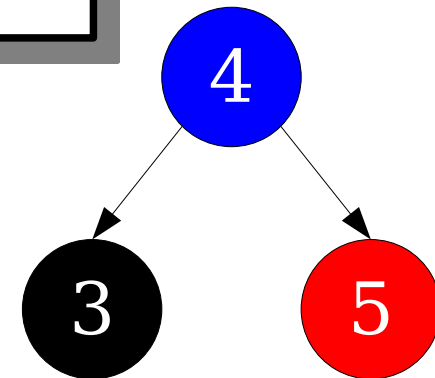
apply
rotation



apply
rotation

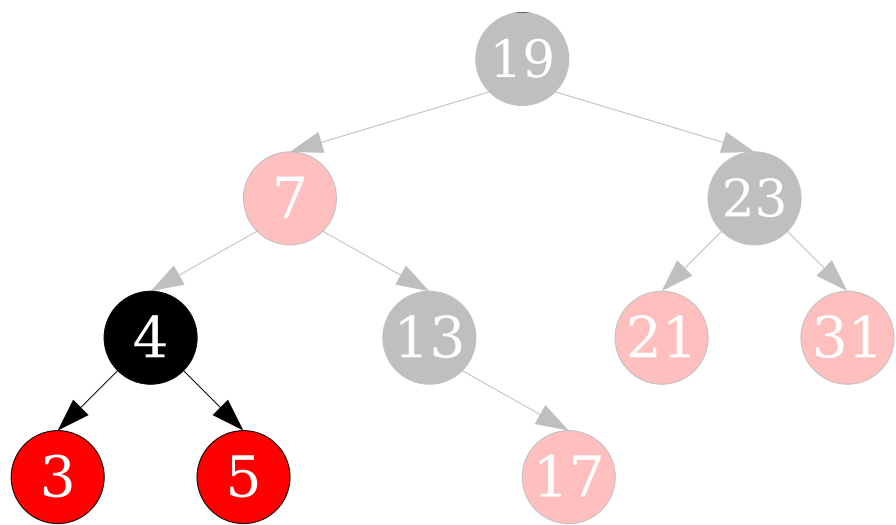
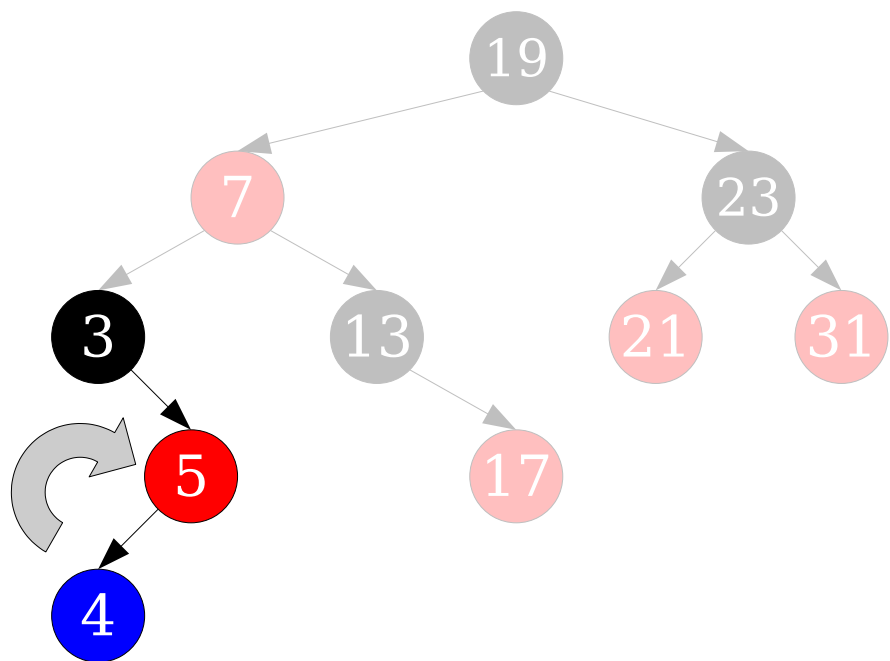


change
colors

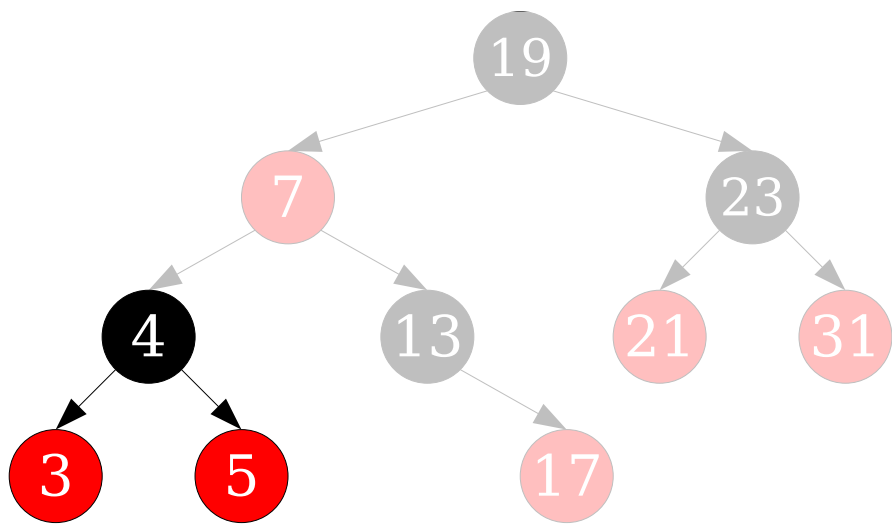
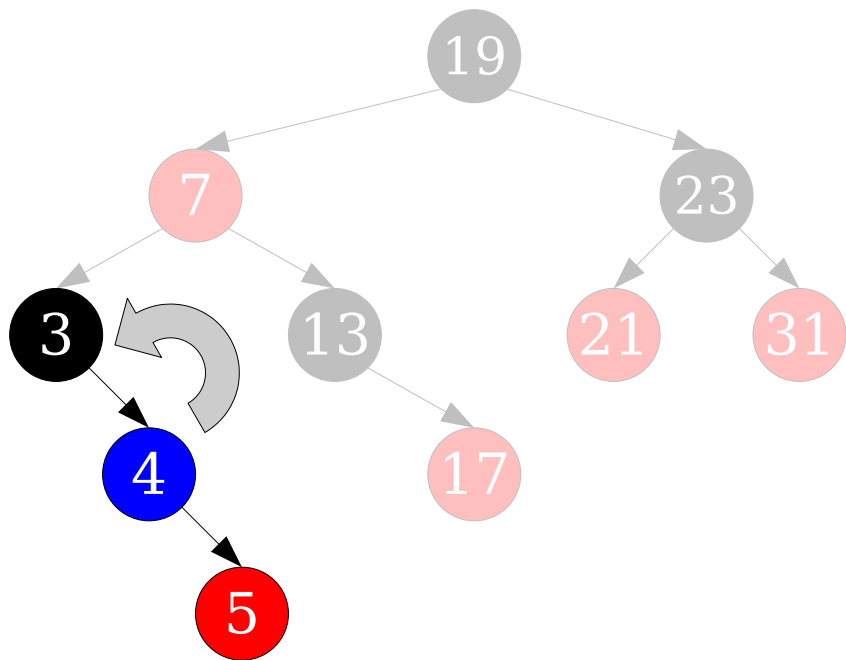


This applies any time we're
inserting a new node into
the middle of a "3-node" in
this pattern.

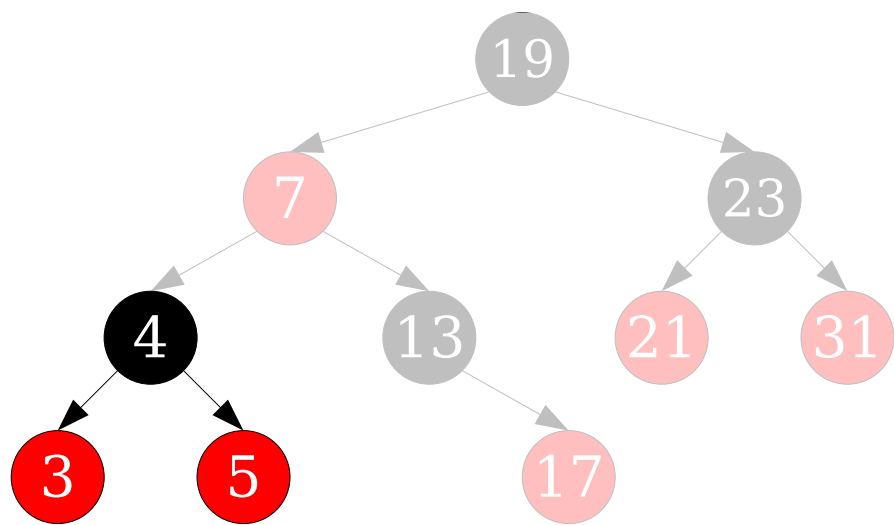
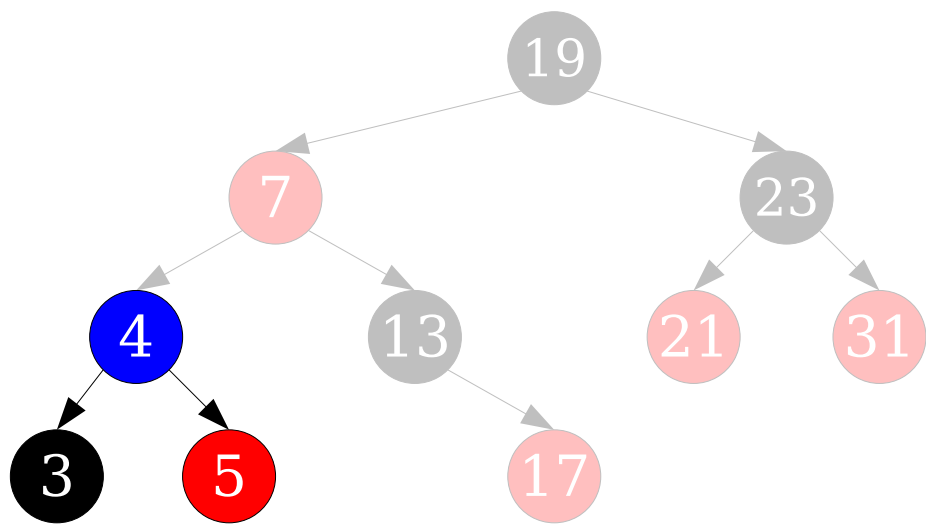
By making observations like
these, we can determine
how to update a red/black
tree after an insertion.



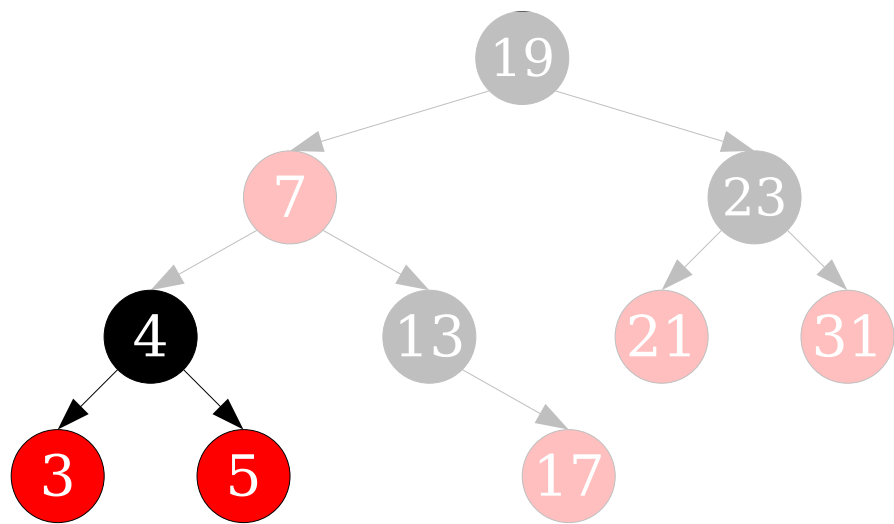
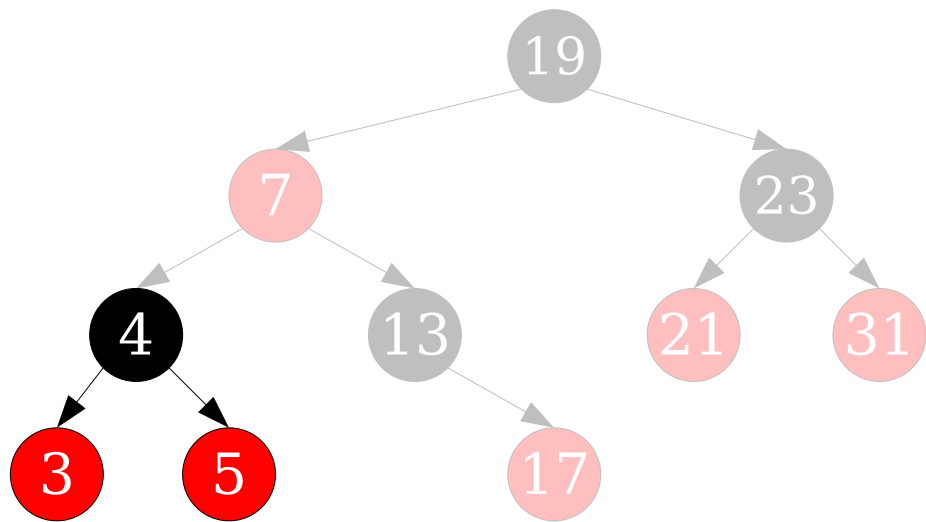
Goal



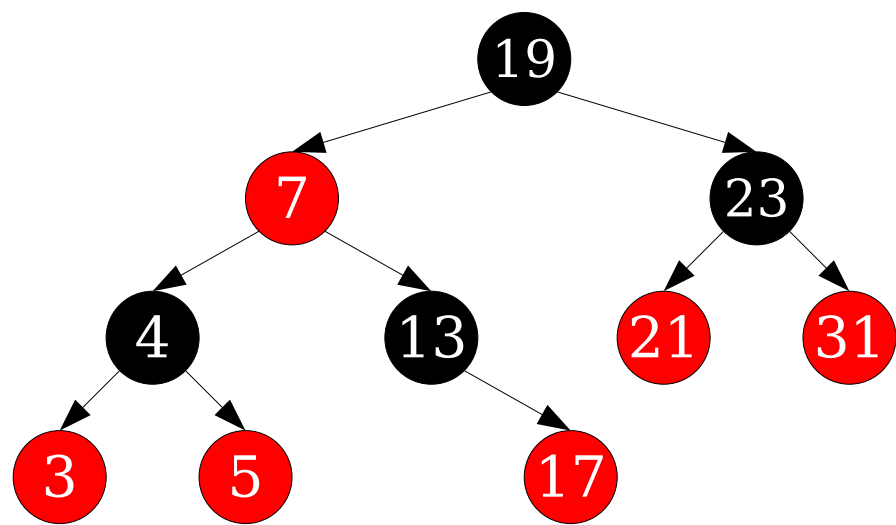
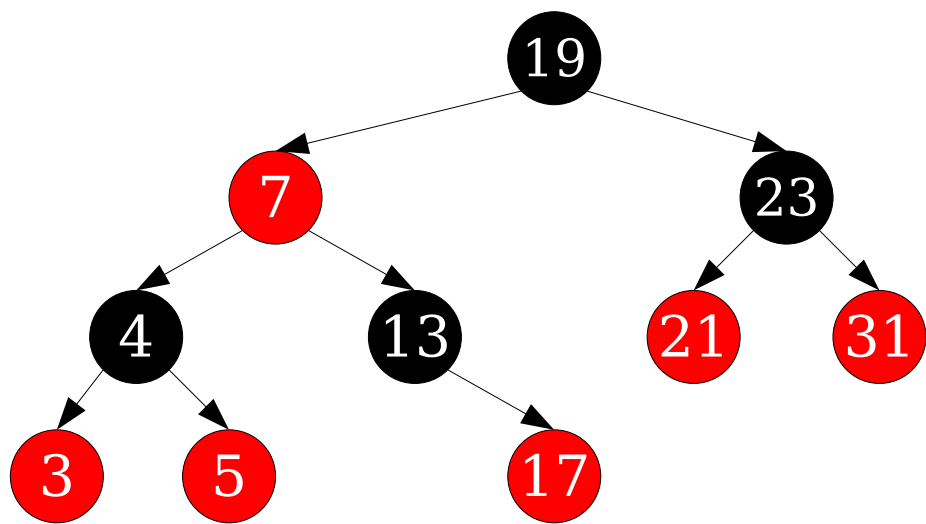
Goal



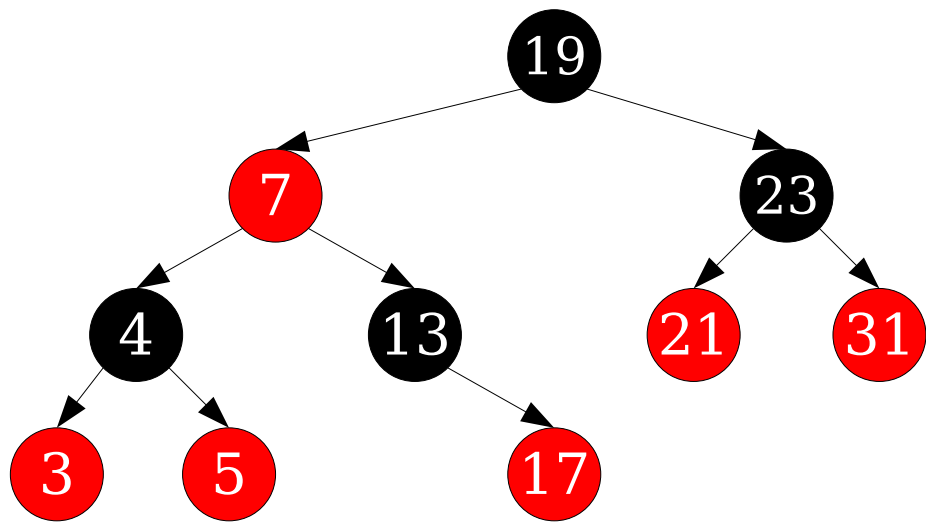
Goal

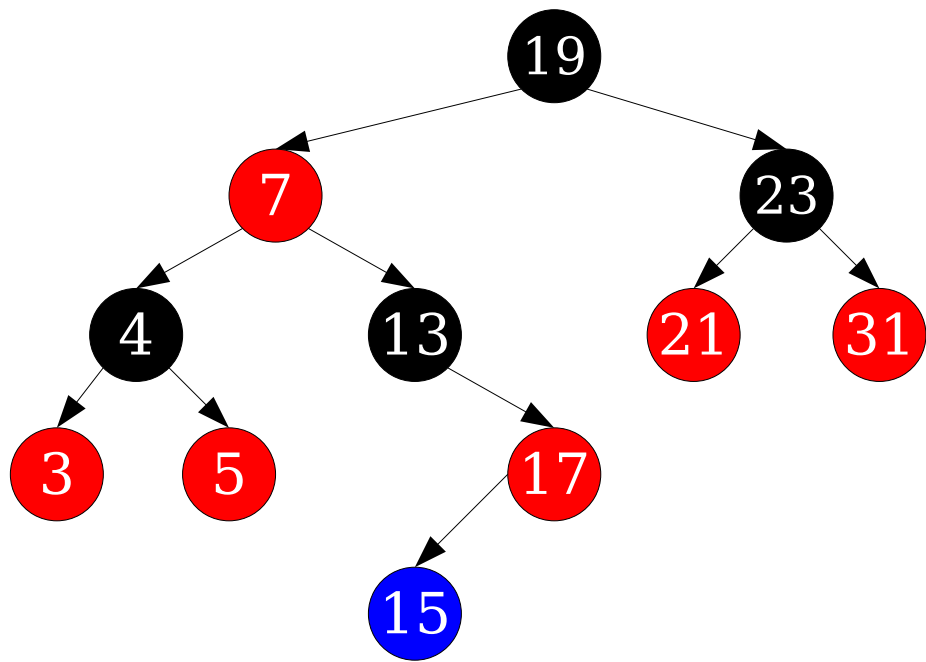


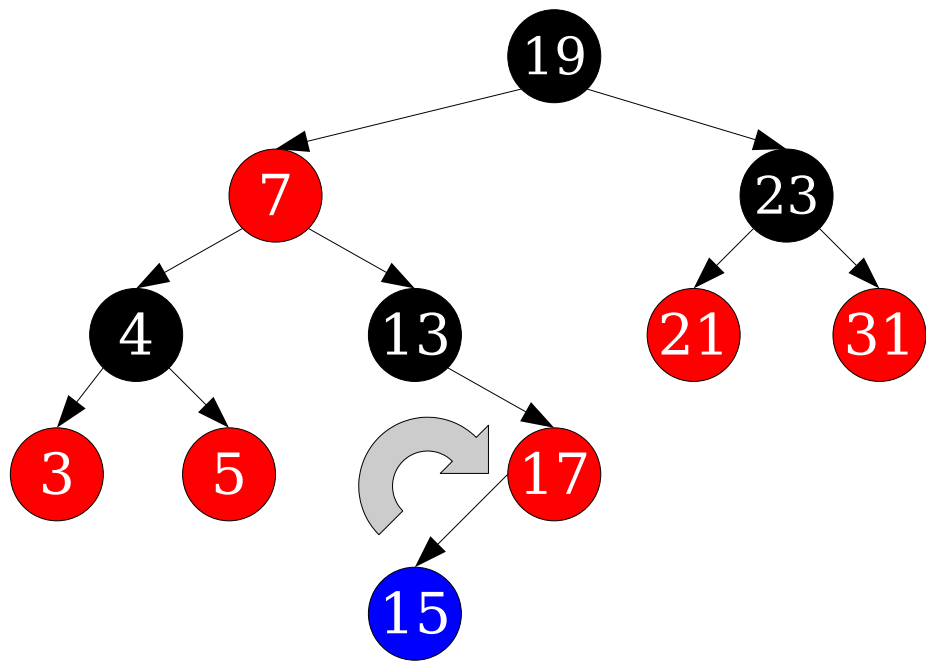
Goal

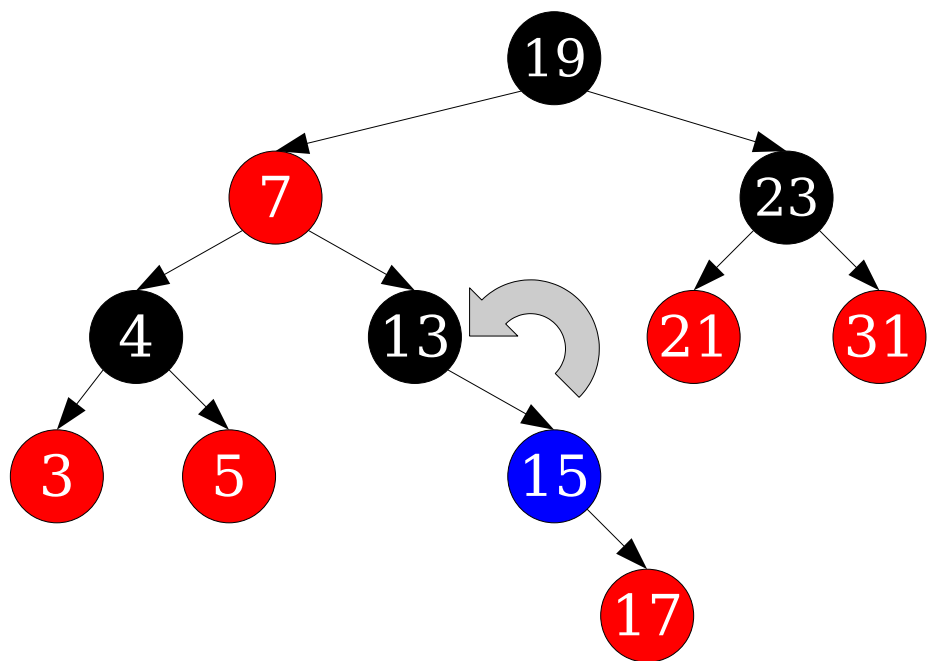


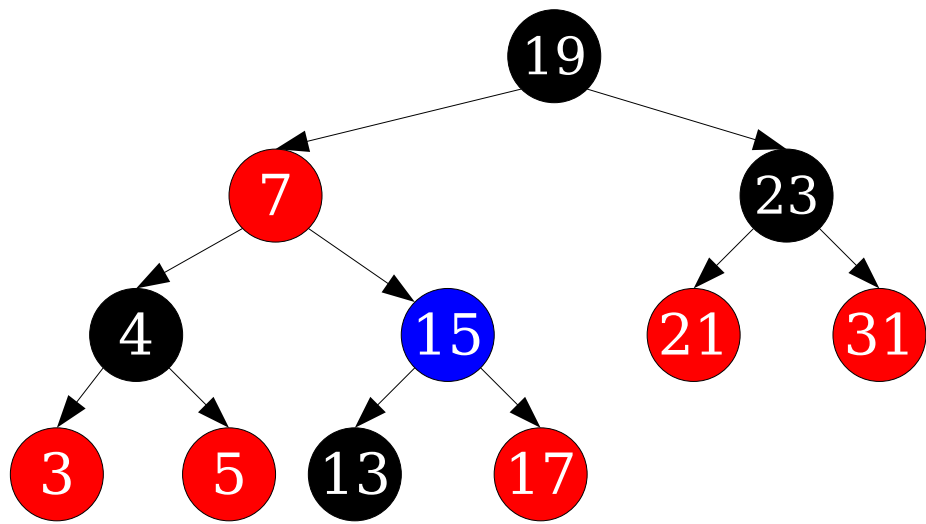
Goal

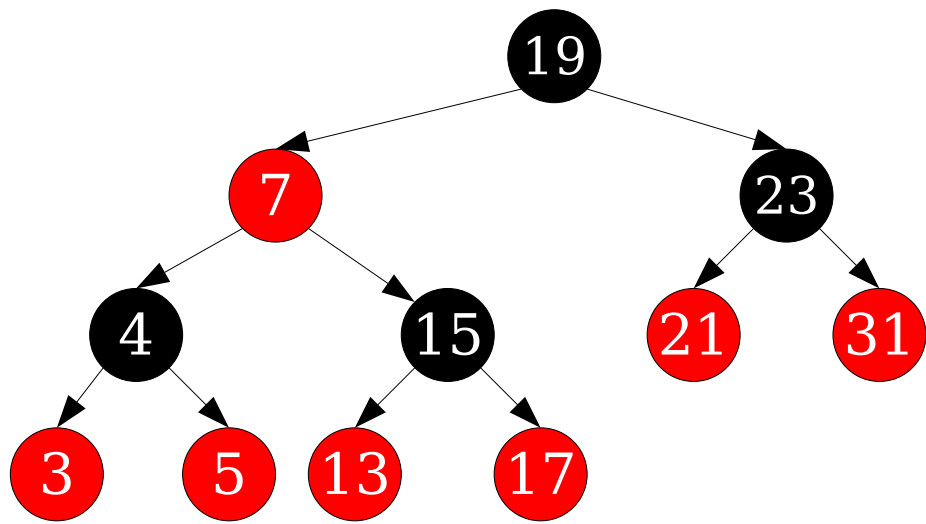


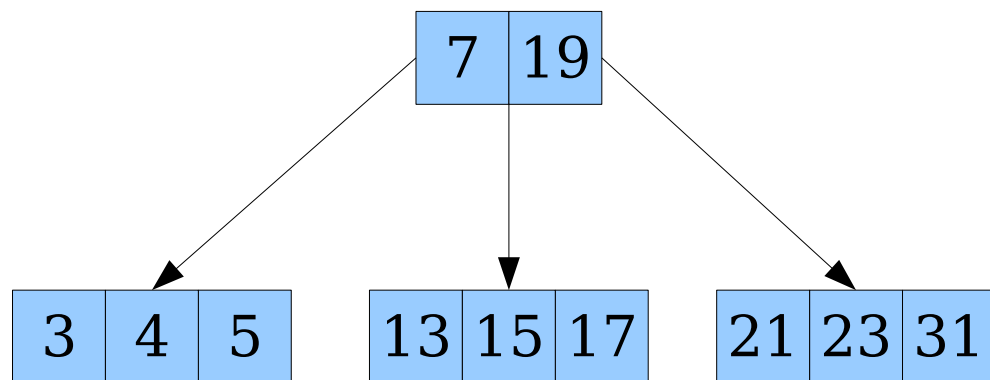
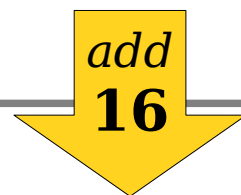
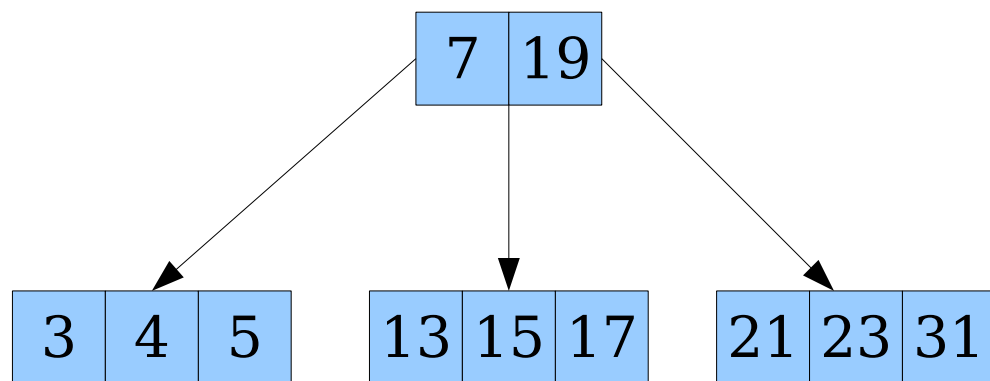
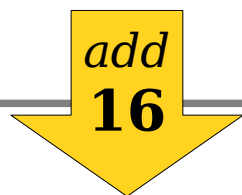
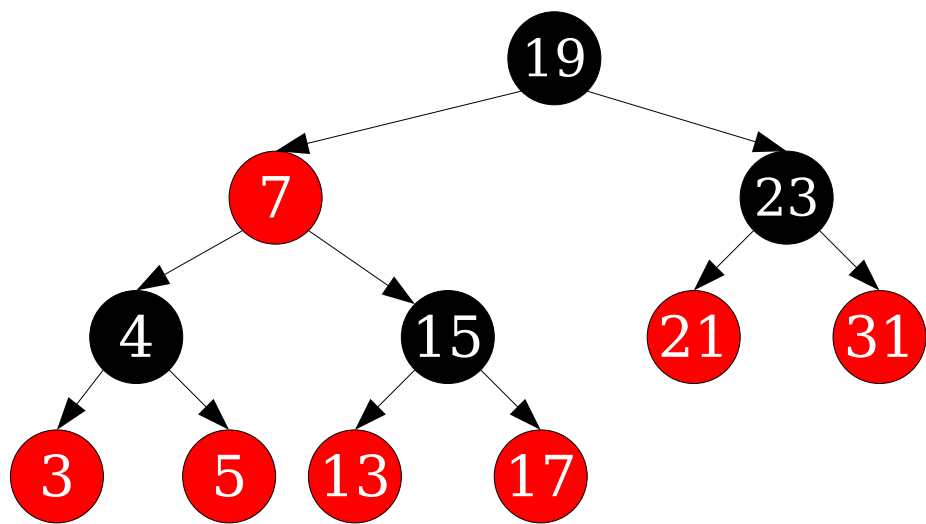


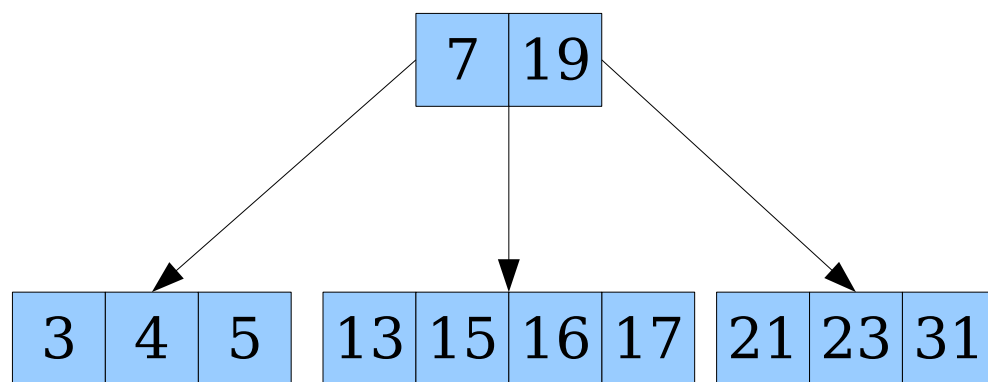
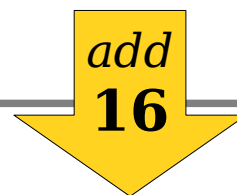
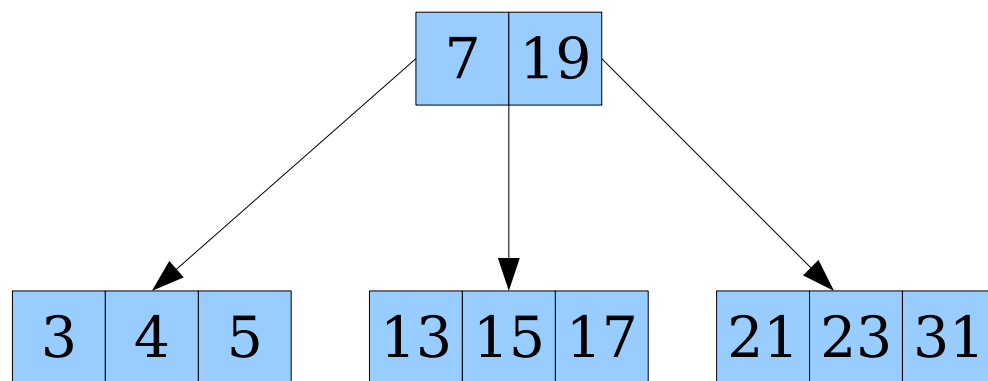
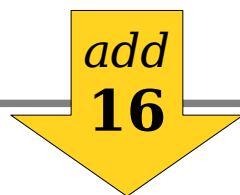
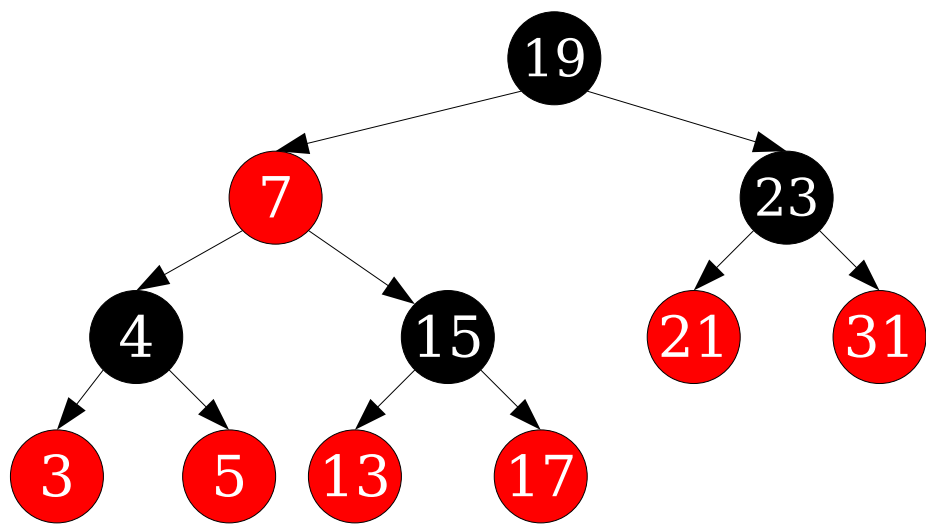


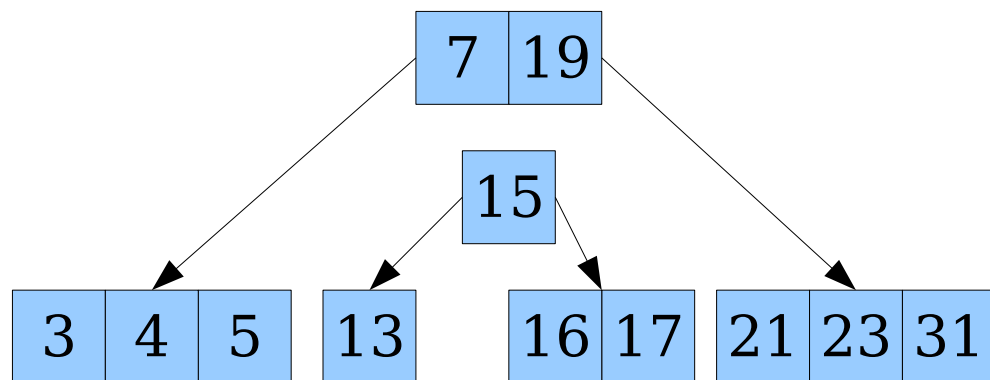
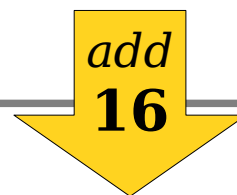
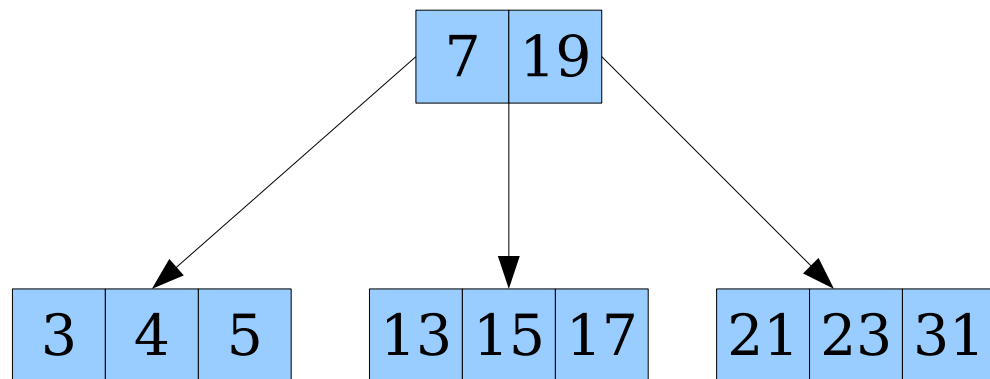
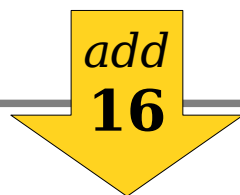
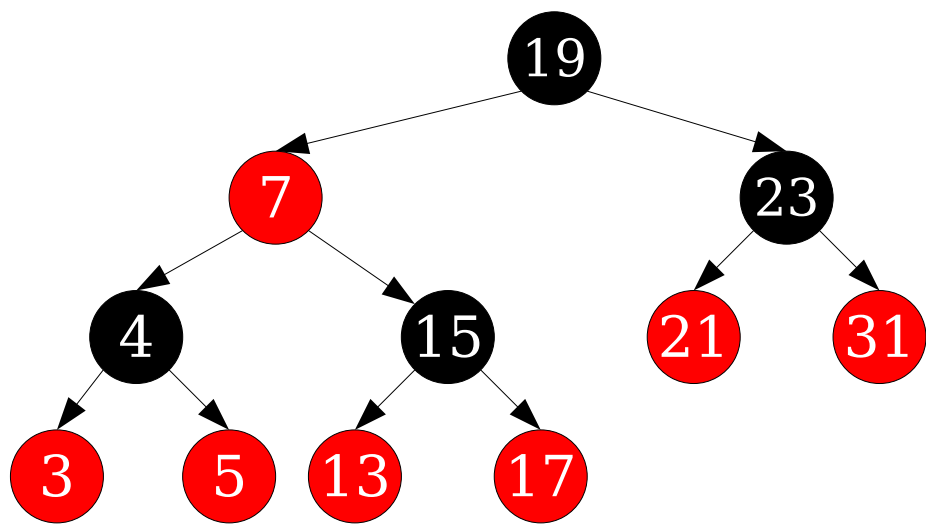


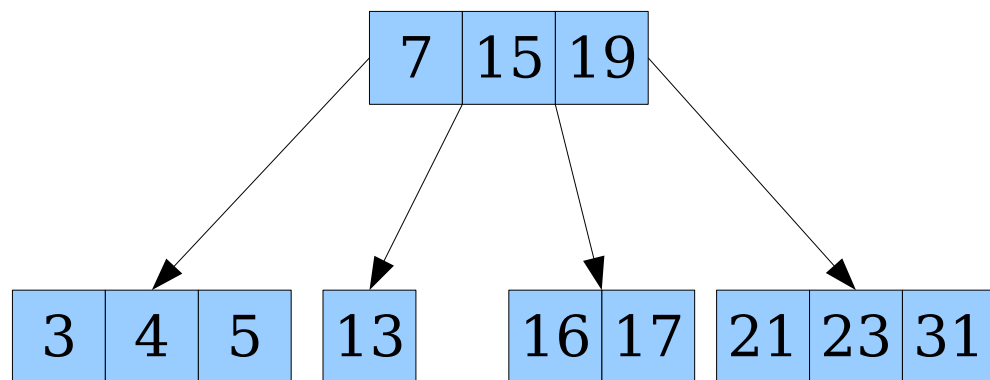
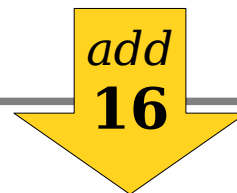
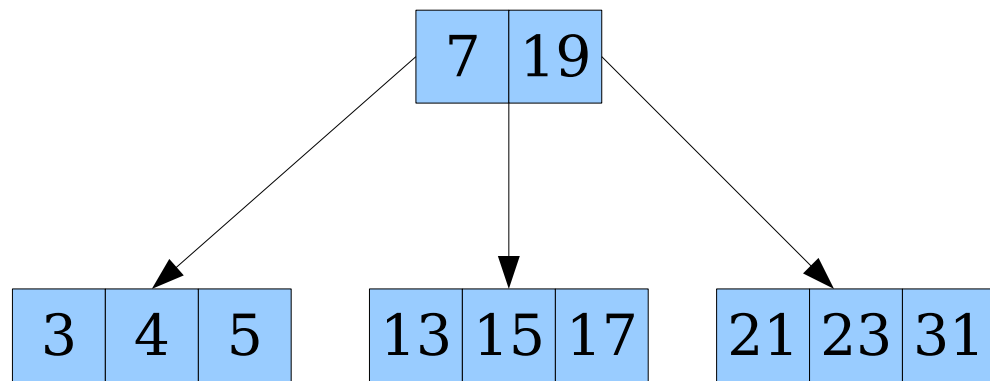
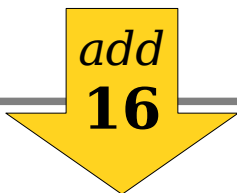
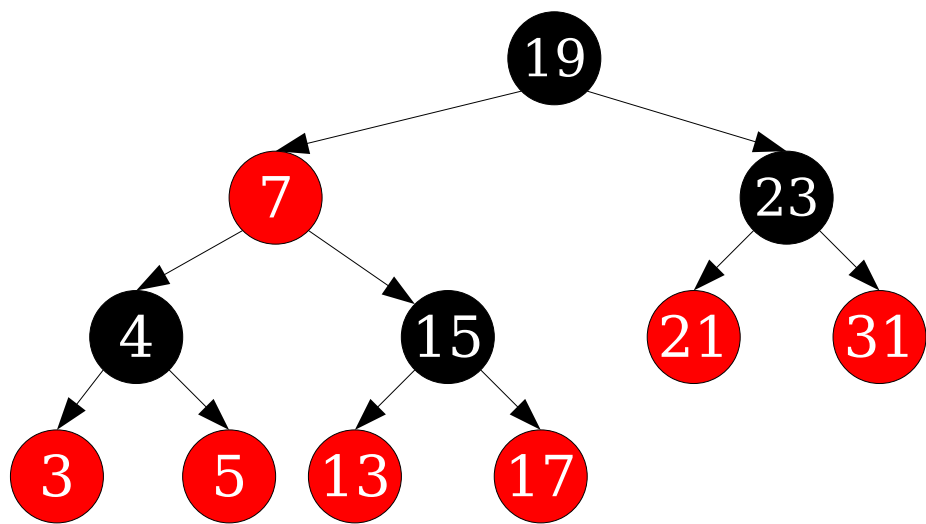


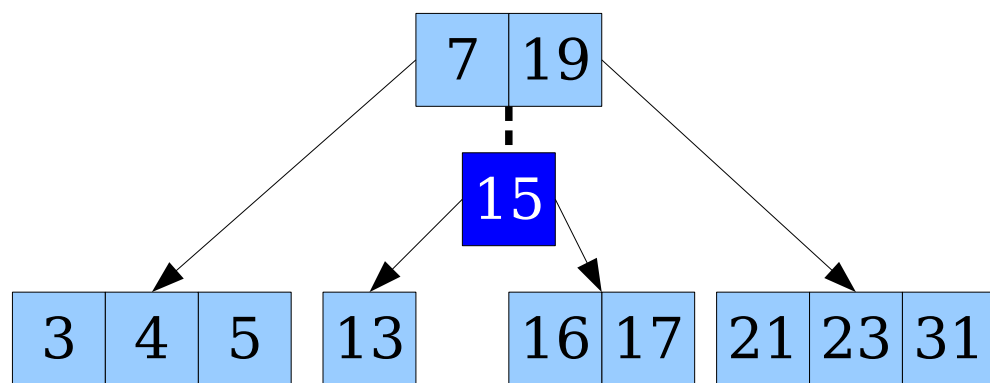
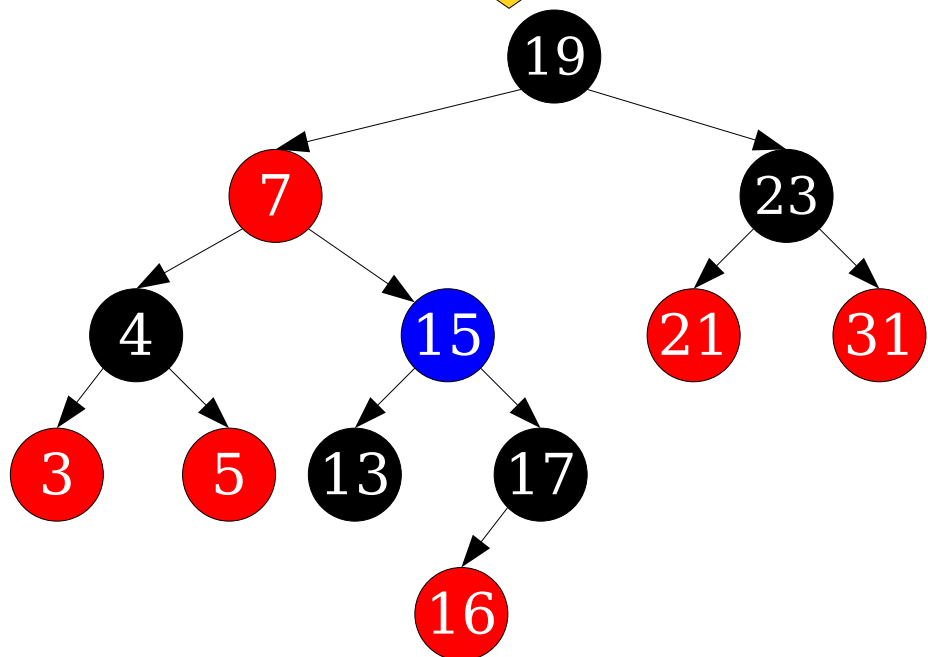
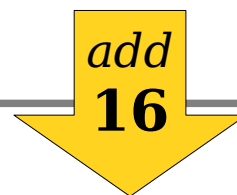
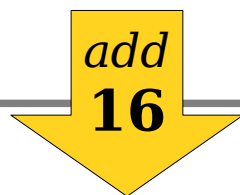
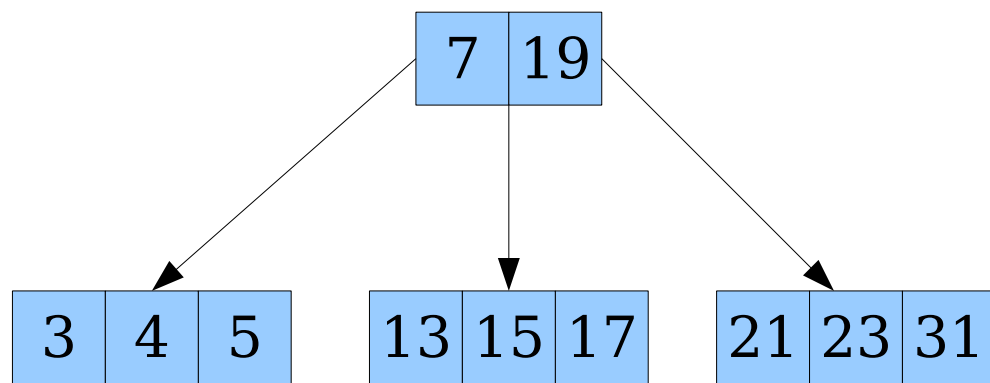
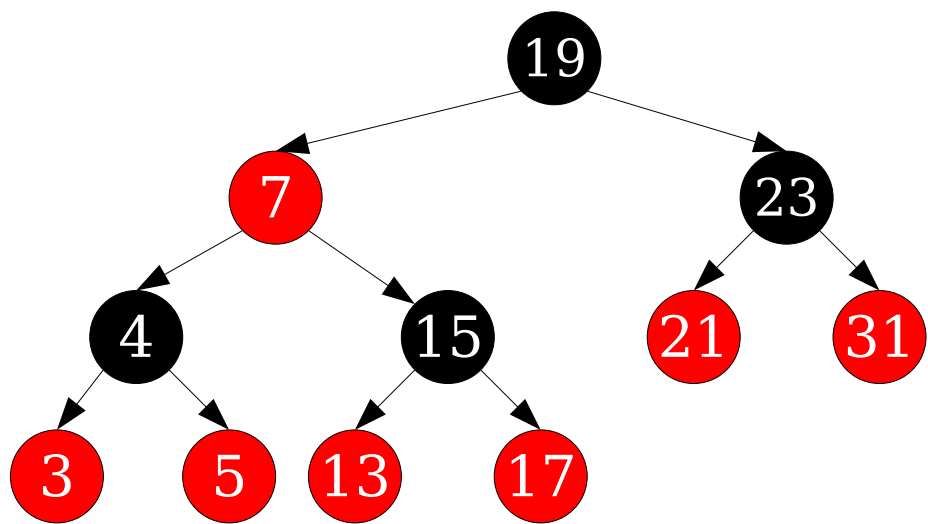


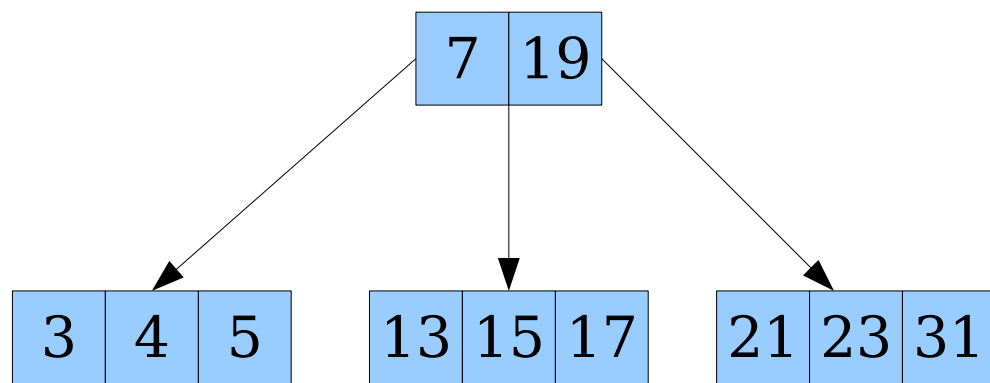
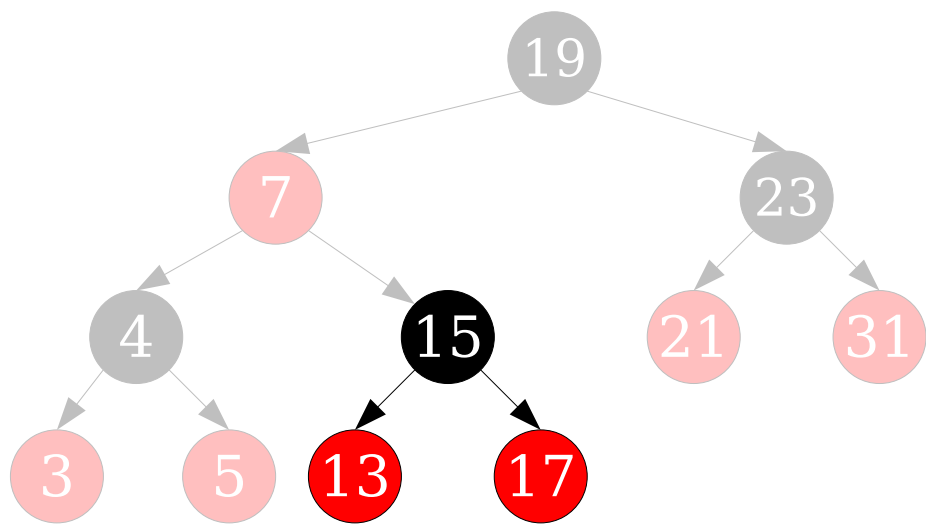






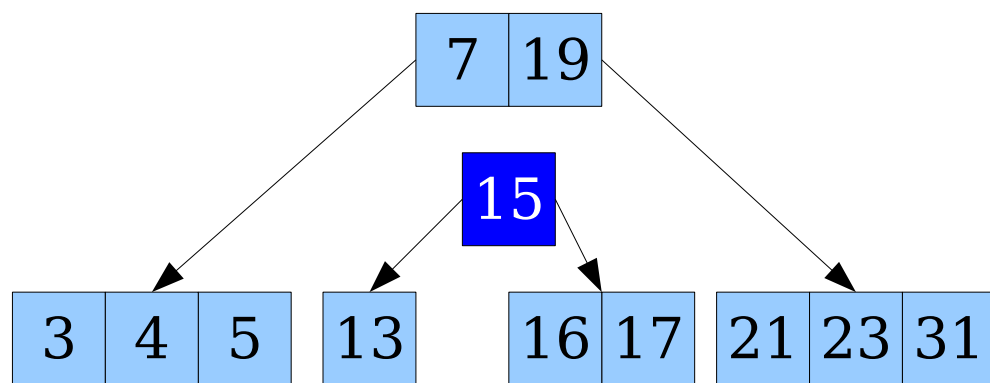
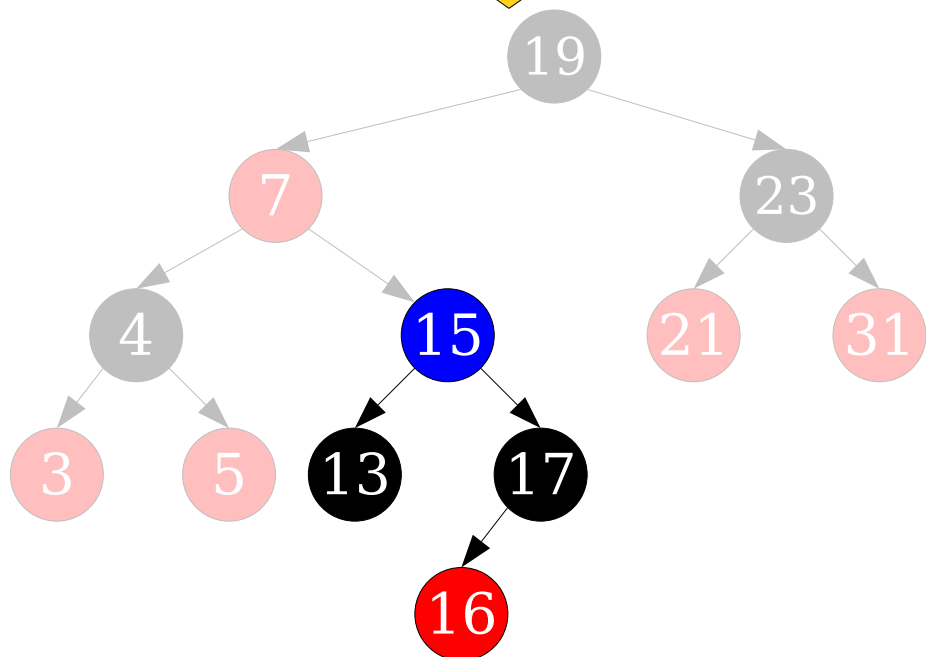


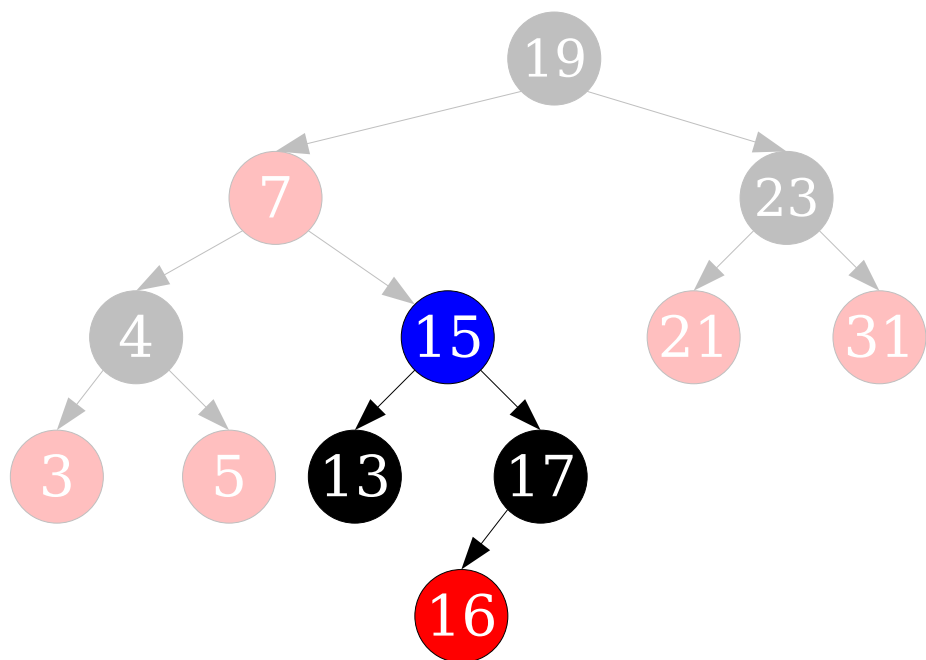
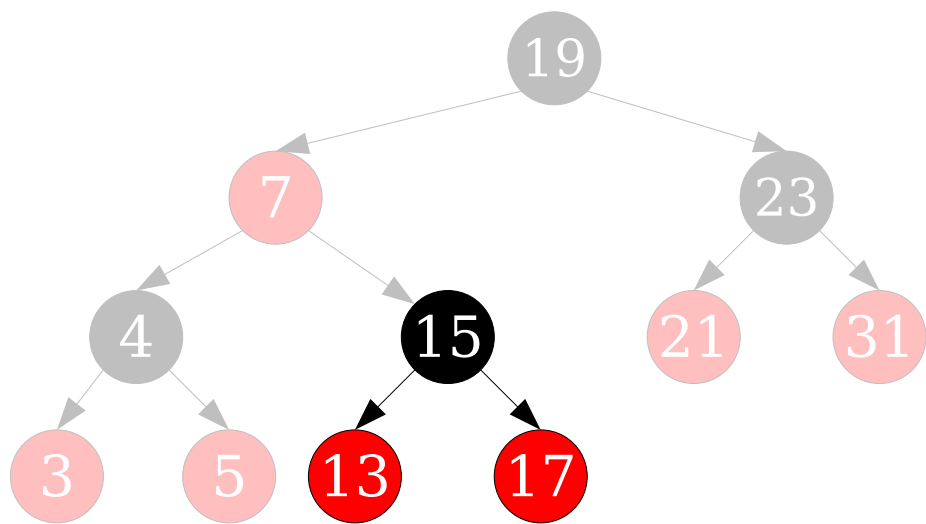


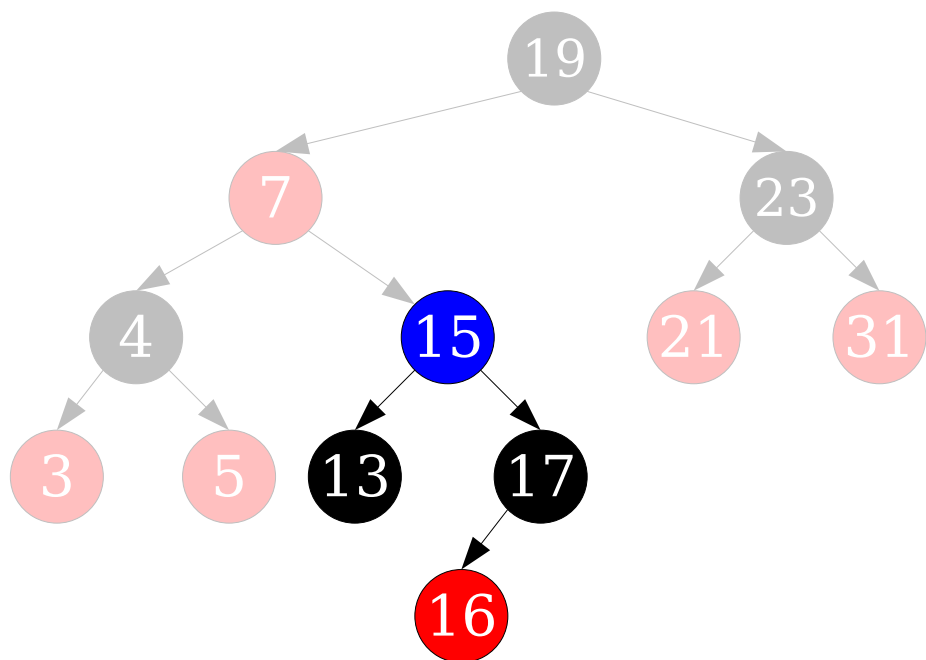
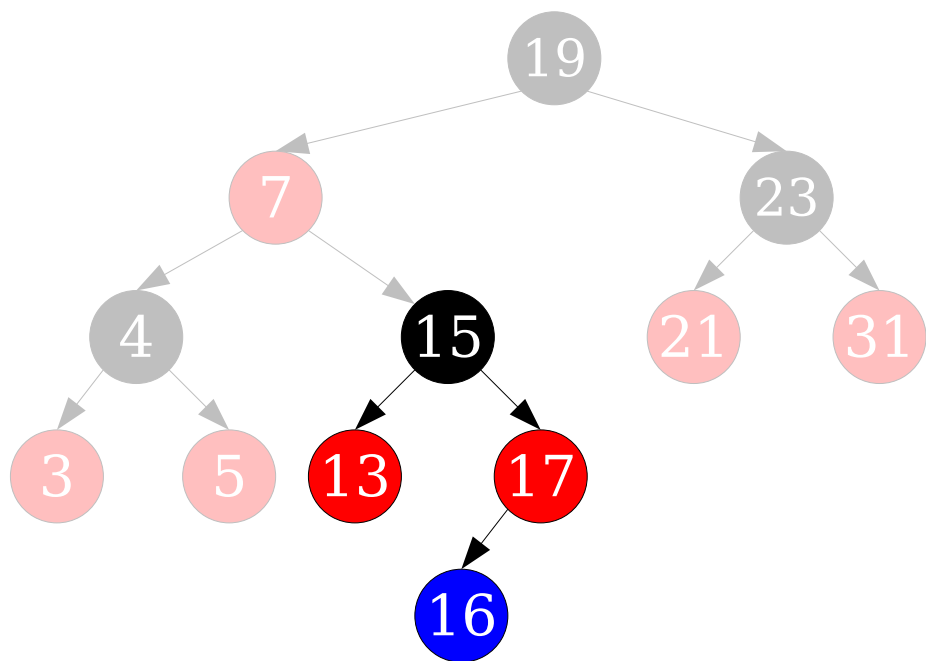


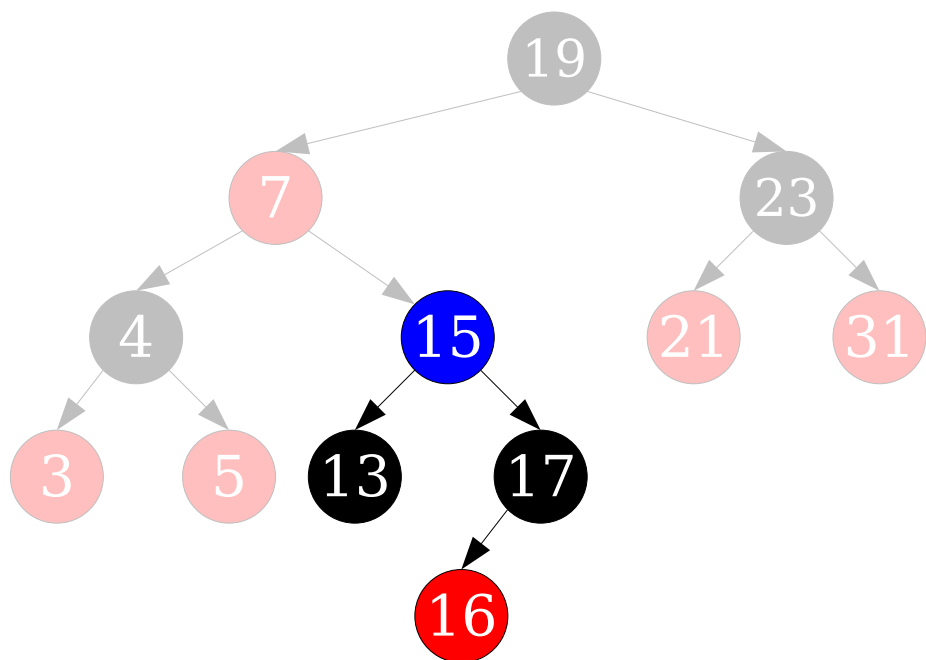
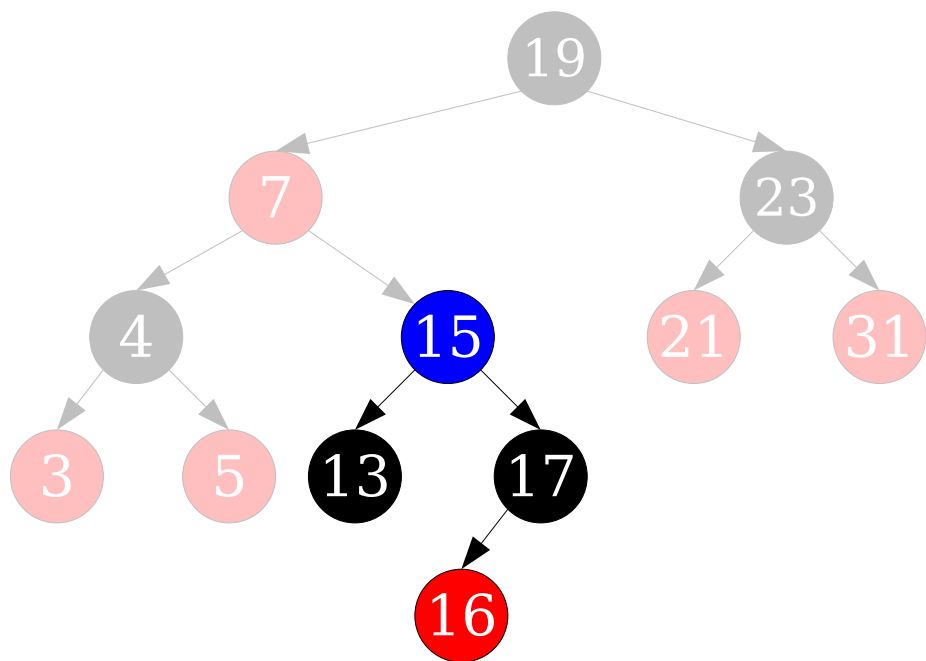
add
16

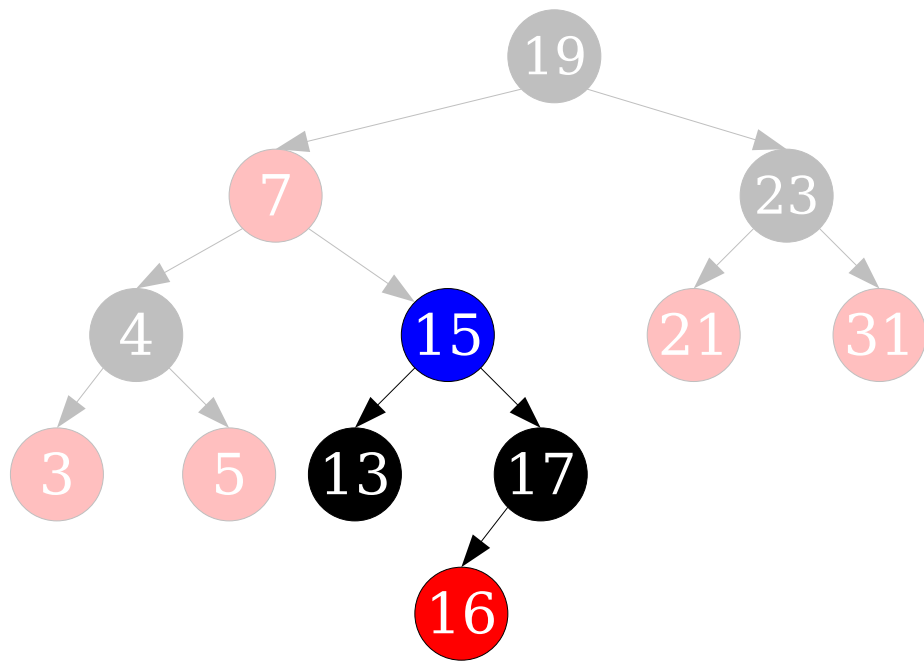
add
16

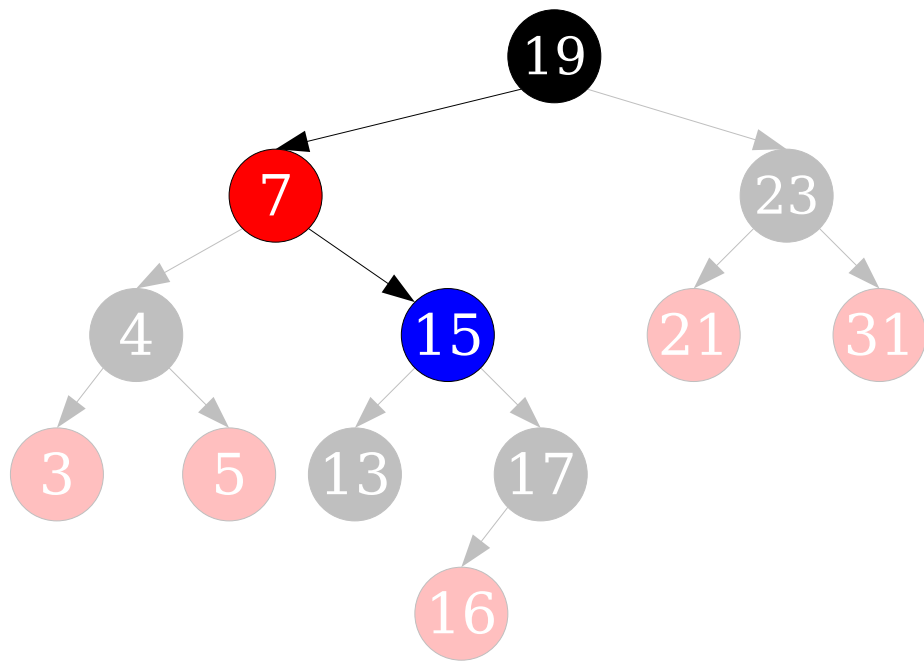


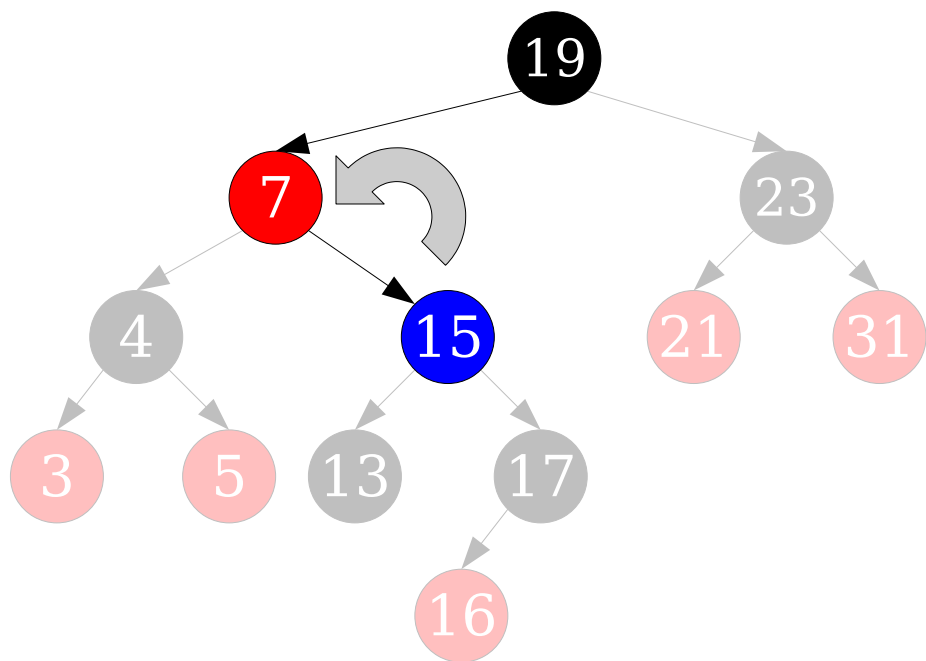


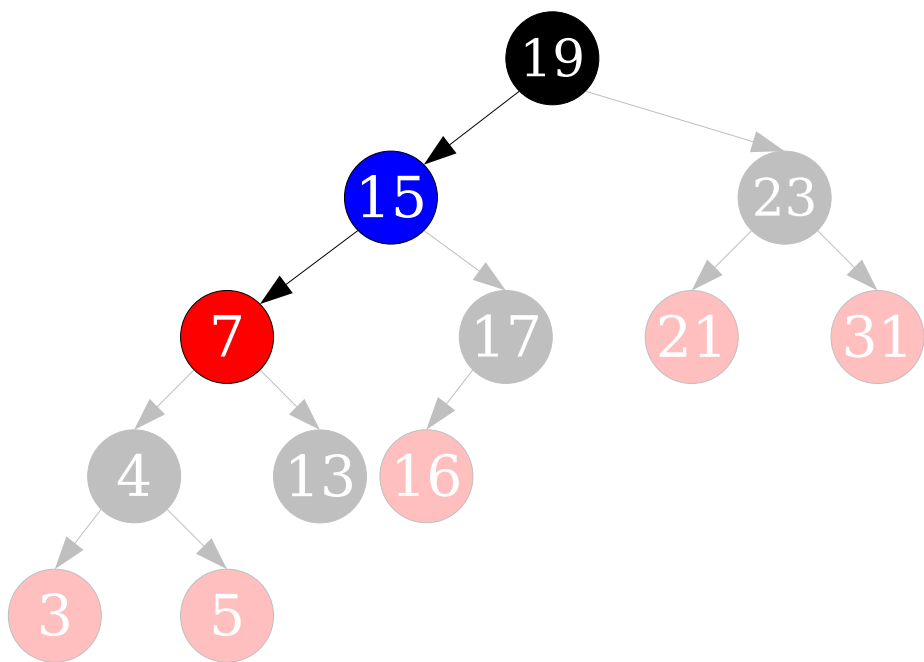


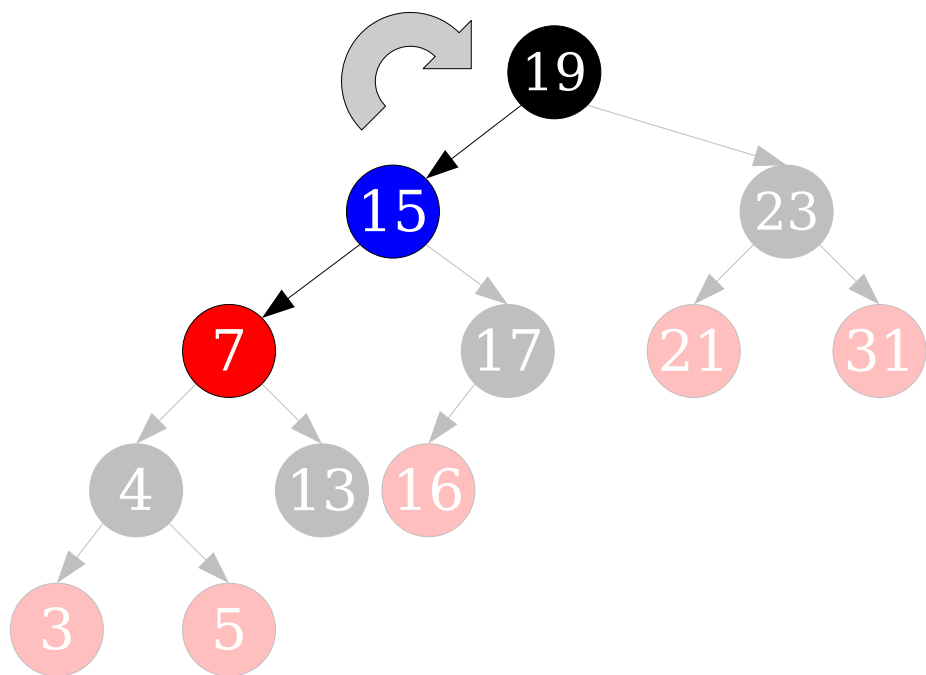


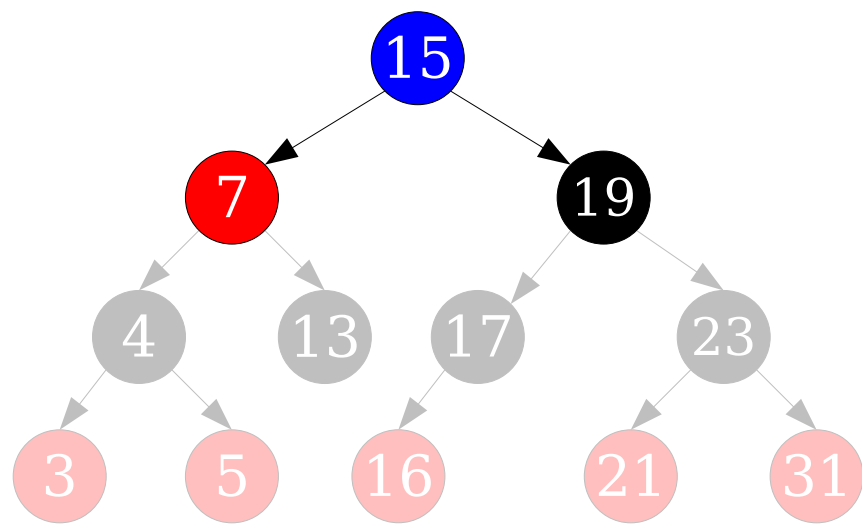


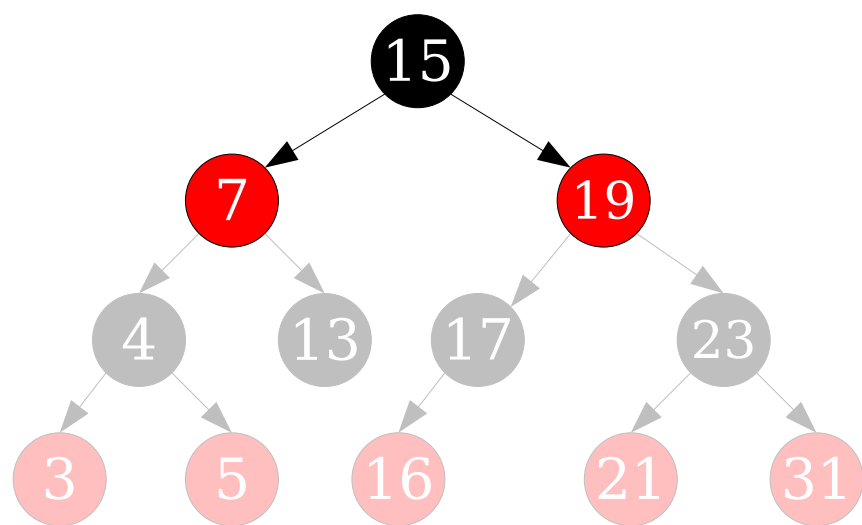


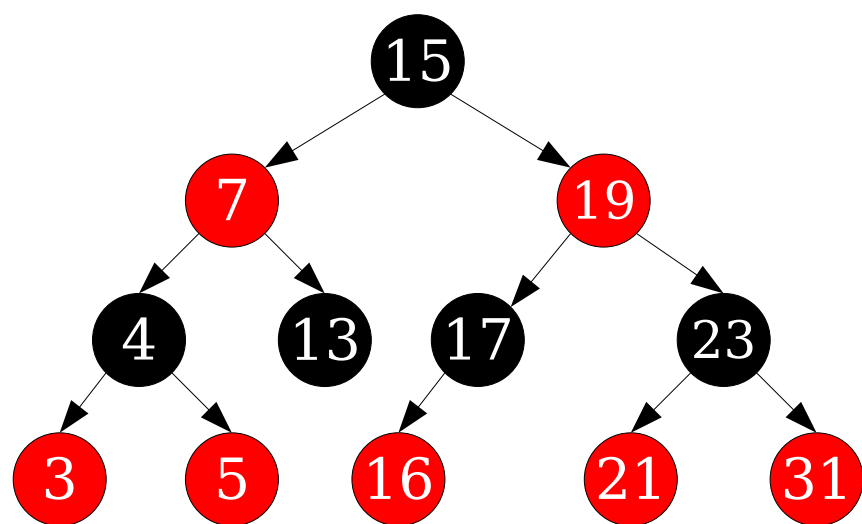












Building Up Rules

- The complex rules on red/black trees make perfect sense if you connect it back to 2-3-4 trees.
- There are lots of cases to consider because there are many different ways you can insert into a red/black tree.
- **Main point:** Simulating the insertion of a key into a node takes time $O(1)$ in all cases. Therefore, since 2-3-4 trees support $O(\log n)$ insertions, red/black trees support $O(\log n)$ insertions.
- The same is true of deletions.

My Advice

- **Do** know how to do B-tree insertions and searches.
 - You can derive these easily if you remember to split nodes.
- **Do** remember the rules for red/black trees and B-trees.
 - These are useful for proving bounds and deriving results.
- **Do** remember the isometry between red/black trees and 2-3-4 trees.
 - Gives immediate intuition for all the red/black tree operations.
- **Don't** memorize the red/black rotations and color flips.
 - This is rarely useful. If you're coding up a red/black tree, just flip open CLRS and translate the pseudocode. ☺